



COE-841

MAPPING AND LOCALIZATION

Jonas T. Ulbrich
Rickson Monteiro

Supervised by
Prof. Ramon Romankevicius Costa

September 23, 2026
Rio de Janeiro

Trabalho 5

September 23, 2026

1 Trabalho 5

This work explores the mapping algorithm for laser-ranger type sensors, which can be extended to lidars as well. The main issue with these algorithms is efficiency, in terms of computation and memory. Indeed, rasterization is required to map the environment and the brute force approach algorithm swipes over the whole raster to update the likelihood of a small number of map-pixels.

```
[1]: %load_ext autoreload
      %autoreload 2

[ ]: import matplotlib.pyplot as plt
      import numpy as np

      from IPython.display import Image
      from matplotlib.animation import FuncAnimation

      from robotics.geometry import Segment, Pose, Angle
      from robotics.robot import Robot
      from robotics.maps import ObstacleMap, OccupancyGrid
      from robotics.range_finder import RangeFinders
      from robotics.control import VelocityControl, generateConstantVelocityTrajectory
```

1.1 Initialize the Maps

In this example we work with a groundtruth / actual environment that is defined with segments, to improve the raycasting's efficiency, as presented in the previous work. The occupancy grid is for memory efficiency a dictionary of points, paired to a likelihood value, where only the points with the likelihood different from the unexplored / initial value are stored.

```
[3]: x_roi = (-5,5)
      y_roi = (-5,5)
      #initialize environment and map
      env = ObstacleMap(
          [
              #limits
              Segment((x_roi[0], y_roi[0]), (x_roi[1], y_roi[0])),
              Segment((x_roi[0], y_roi[0]), (x_roi[0], y_roi[1])),
              Segment((x_roi[1], y_roi[1]), (x_roi[1], y_roi[0])),
```

```

        Segment((x_roi[1], y_roi[1]), (x_roi[0], y_roi[1])),

        Segment.horizontal(-5,-2,4),
        Segment.vertical(-1,-5,3),

        Segment.vertical(2,-5,1),
        Segment((2,-4), (4,-2)),
        Segment.vertical(4,-2,2),
        Segment((4,0), (2,2)),
        Segment((2,2), (-2,2)),
        Segment.vertical(-2,2,3),

    ],
    x_lims=(-5, 5),
    y_lims=(-5, 5)
)

o_grid_resolution = 0.1
o_grid = OccupancyGrid(x_roi, y_roi, o_grid_resolution)

```

1.2 Simulation Parameters

```

[4]: #set initial position
x0 = Pose(0, 0., Angle.from_deg(10.))

#set dt for simulation
dt = 0.05

```

1.3 Deinition of the Sensor

In this example we implement a very rudimentary collision avoidance algorithm that makes use of the sensor's readings. It implements a P-Controller that adjusts the ω and v velocities of the control input, before applying it.

```

[5]: #define sensor:
front_left   = Pose(0, 0.5, Angle(np.pi/6))
front_right  = Pose(0, -0.5, Angle(- np.pi/6))
front_center = Pose(0.5, 0, Angle(0.))
back_left    = Pose(-0.5, 0, Angle(np.pi - np.pi/6))
back_right   = Pose(-0.5, 0, Angle(np.pi + np.pi/6))

range_finder = RangeFinders(
    [
        front_left,
        front_right,
        front_center,
        back_left,

```

```

        back_right,
    ],
    beam_fov=Angle.from_deg(2),
    robot_pose=x0
)

min_dist_to_wall = 1
v_max = 2
w_max = np.pi / 4
w_rotate = np.pi / 6
rotate_angle_when_in_front_of_obstacle = np.pi / 8
rot_time = rotate_angle_when_in_front_of_obstacle / w_rotate
rot_sign = 1
iter_to_rotate = rot_time / dt
rotated_iter = iter_to_rotate

#implement collision avoidance
def collisionAvoidance(u_in: VelocityControl):
    global range_finder, min_dist_to_wall, v_max, w_max,
    ↪rotate_angle_when_in_front_of_obstacle, rotated_iter, iter_to_rotate,
    ↪w_rotate, rot_sign

    u = u_in.copy() #avoid modifying the original

    k_fl = 10
    k_bl = 0.1
    k_fr = 10
    k_br = 0.
    k_fc = 1.2
    k_bc = 0.

    min_dist_to_rot = 0.1

    def zero_clip(val, clip_val):
        if val > clip_val: return 0
        else: return val

    if rotated_iter < iter_to_rotate:
        u.w = w_rotate
        u.v = 0
        rotated_iter += 1
    else:
        fl = zero_clip(range_finder.z_rangers[0].reading.r, min_dist_to_wall)
        fr = zero_clip(range_finder.z_rangers[1].reading.r, min_dist_to_wall)
        fc = zero_clip(range_finder.z_rangers[2].reading.r, min_dist_to_wall)
        bl = zero_clip(range_finder.z_rangers[3].reading.r, min_dist_to_wall)
        br = zero_clip(range_finder.z_rangers[4].reading.r, min_dist_to_wall)

```

```

        bc = zero_clip(np.sqrt(br**2 + bl**2) / 2, min_dist_to_wall)

        fl_is_close_to_obj = (fl > 1e-6 and fl < 0.5 * np.cos(np.pi/6) +
↪min_dist_to_rot)
        fr_is_close_to_obj = (fr > 1e-6 and fr < 0.5 * np.cos(np.pi/6) +
↪min_dist_to_rot)
        fc_is_close_to_obj = (fc > 1e-6 and fc < min_dist_to_rot)

        if fl_is_close_to_obj or fr_is_close_to_obj or fc_is_close_to_obj:
            rotated_iter = 0
            u.w = w_rotate
            u.v = 0

            return u

        fl = min_dist_to_wall - fl if fl > 0. else 0
        fr = min_dist_to_wall - fr if fr > 0. else 0
        fc = min_dist_to_wall - fc if fc > 0. else 0
        bl = min_dist_to_wall - bl if bl > 0. else 0
        br = min_dist_to_wall - br if br > 0. else 0
        bc = min_dist_to_wall - bc if bc > 0. else 0

        turn_right = k_fl * fl + k_bl * bl
        turn_left  = k_fr * fr + k_br * br

        w_corr = turn_left - turn_right
        v_corr = k_bc * bc - k_fc * fc

        v_new = np.clip(u.v + v_corr, 0., v_max)
        w_new = np.clip(u.w + w_corr, -w_max, w_max)

        if v_new < 1e-6 and u.v > 1e-6:
            rotated_iter = 0
            u.w = w_rotate
            u.v = 0
        else:
            u.w = w_new
            u.v = v_new

    return u

```

1.4 Initialization of the Robot

The robot is initialized at given initila position, with the sensor defined above and the collision avoidance algorithm

```
[6]: #define the robot
robot = Robot(
    x0,
    sensors = [
        range_finder
    ],
    collisionAvoidance = collisionAvoidance
)
```

2 Plot of the initial state

The robot in its initial state is shown, along with the range finder's measurements. Furthermore the updated occupancy grid after the first taken measurement

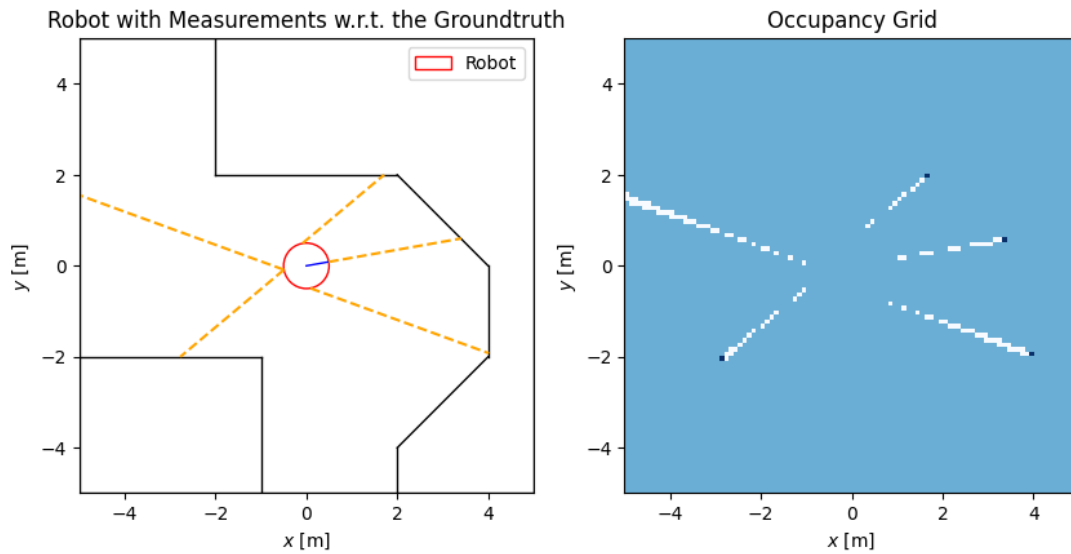
```
[7]: robot.measure(env)
range_finder.update_occupancy_grid(o_grid)

#plot initial state, with first measurement
f = plt.figure(figsize=(10, 5))
f.suptitle("Initial State")

ax1 = plt.subplot(1,2, 1)
ax1.set_title("Robot with Measurements w.r.t. the Groundtruth")
ax1.set_xlabel("$x$ [m]")
ax1.set_ylabel("$y$ [m]")
env.draw(ax1)
robot.draw(ax1, r = 0.5)

ax2 = plt.subplot(1,2, 2)
ax2.set_title("Occupancy Grid")
ax2.set_xlabel("$x$ [m]")
ax2.set_ylabel("$y$ [m]")
o_grid.drawImg(ax2)
```

Initial State



```
[8]: animation_time = 30 #[s]

n_frames = int(animation_time / dt)
#trajectory = generateRandomVelocityTrajectory(n_frames,dt)
trajectory = generateConstantVelocityTrajectory(n_frames, 0.5, 0., dt)
applied_controls = []

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

env.draw(ax1)
ax1.set_aspect('equal', adjustable='box')
ax1.set_title("Robot with Measurements w.r.t. the Groundtruth")
ax1.set_xlabel("$x$ [m]")
ax1.set_ylabel("$y$ [m]")

ax2.set_title("Occupancy Grid")
ax2.set_xlabel("$x$ [m]")
ax2.set_ylabel("$y$ [m]")
#update function for animation
def update(frame):
    global applied_controls
    ax2.clear()
    ax1.clear()

    u = trajectory[frame]
```

```

    applied_controls.append(robot.applyControl(u))

    robot.measure(env)
    range_finder.update_occupancy_grid(o_grid)

    env.draw(ax1)
    robot.draw(ax1, r = 0.5)

    o_grid.drawImage(ax2)

ani = FuncAnimation(fig, update, frames=np.arange(0, n_frames), interval=50)

gif_path = 'img/knownCorr.gif'
ani.save(gif_path, writer='pillow', fps=20)

plt.close(fig)

display(Image(filename=gif_path))

```

<IPython.core.display.Image object>

```

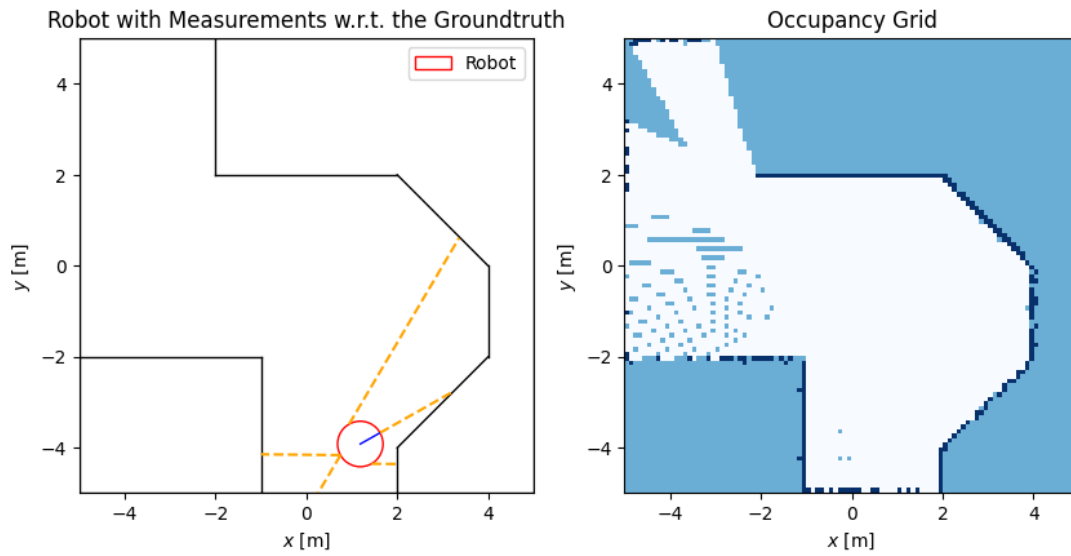
[9]: #plot final state, with first measurement
f = plt.figure(figsize=(10, 5))
f.suptitle("Final State")

ax1 = plt.subplot(1,2, 1)
ax1.set_title("Robot with Measurements w.r.t. the Groundtruth")
ax1.set_xlabel("$x$ [m]")
ax1.set_ylabel("$y$ [m]")
env.draw(ax1)
robot.draw(ax1, r = 0.5)

ax2 = plt.subplot(1,2, 2)
ax2.set_title("Occupancy Grid")
ax2.set_xlabel("$x$ [m]")
ax2.set_ylabel("$y$ [m]")
o_grid.drawImage(ax2)

```

Final State



```
[11]: v_traj= []
w_traj = []
v_eff_traj = []
w_eff_traj = []
t_traj = []

t = 0
for u, u_eff in zip(trajectory, applied_controls):
    t_traj.append(t)
    v_traj.append(u.v)
    w_traj.append(u.w)
    v_eff_traj.append(u_eff.v)
    w_eff_traj.append(u_eff.w)
    t += dt

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))
ax1.set_title("Trajectory for $v$")
ax1.set_xlabel("Time [s]")
ax1.set_ylabel(r"$v$ [m/s]")

ax2.set_title("Trajectory for $\omega$")
ax2.set_xlabel("Time [s]")
ax2.set_ylabel(r"$\omega$ [rad/s]")

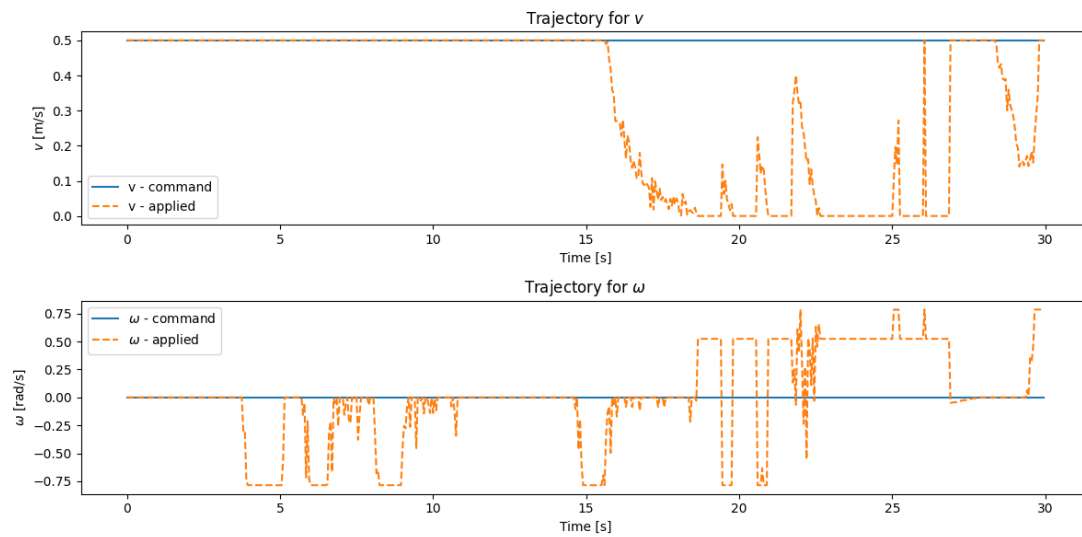
ax1.plot(t_traj, v_traj, label="v - command")
ax1.plot(t_traj, v_eff_traj, linestyle='--', label="v - applied")
```

```

ax1.legend()

ax2.plot(t_traj, w_traj, label="$\\omega$ - command")
ax2.plot(t_traj, w_eff_traj, linestyle='--', label="$\\omega$ - applied")
ax2.legend()
plt.tight_layout()

```



[]:

Trabalho 6

September 23, 2026

1 Trabalho 6

```
[ ]: %load_ext autoreload
      %autoreload 2
```

```
[ ]: import matplotlib.pyplot as plt
      import numpy as np

      from IPython.display import Image
      from matplotlib.animation import FuncAnimation

      from robotics.geometry import Pose, Angle
      from robotics.robot import Robot
      from robotics.maps import LandmarkMap, Landmark
      from robotics.control import generateConstantVelocityTrajectory
      from robotics.ekf import EKFSlamKnownCorrespondences, \
      ↪EKFSlamUnknownCorrespondences
      from robotics.camera import Camera
```

```
[ ]: # create map
      n_landmarks = 15

      x_lims = (-5, 5)
      y_lims = (-5, 5)

      landmarks = []
      for signature in range(n_landmarks):
          #sample coordinates in roi
          x = np.random.uniform(x_lims[0], x_lims[1])
          y = np.random.uniform(y_lims[0], y_lims[1])
          phi = Angle(np.random.uniform(0, 2 * np.pi))
          #create landmark
          landmarks.append(Landmark(Pose(x, y, phi), signature))

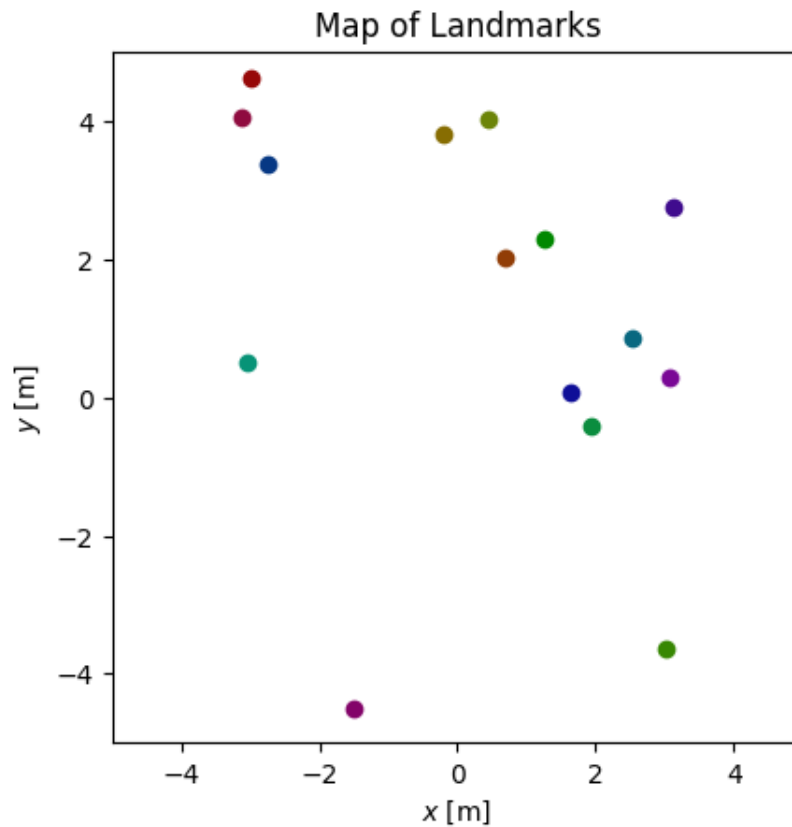
      env = LandmarkMap(landmarks, x_lims=x_lims, y_lims=y_lims)

      #plot map
```

```
f, ax = plt.subplots()

ax.set_title("Map of Landmarks")
ax.set_xlabel("$x$ [m]")
ax.set_ylabel("$y$ [m]")
ax.set_aspect('equal', adjustable='box')

env.draw(ax)
```



```
[ ]: #set initial position
x0 = Pose(0, 0, Angle.from_deg(0))

#set dt for simulation
dt = 0.05 #[s]
```

```
[ ]: fov = Angle.from_deg(80) #[°]
sensor_range = 3 #[m]

#define robot's sensor
```

```

camera = Camera(Pose(0.5, 0, Angle.from_deg(0)), field_of_view=fov,
↳ sensor_range=sensor_range, robot_pose=x0)
measurement_frequency = 10 #[Hz]

```

```

[ ]: #define the robot
robot = Robot(
    x0,
    sensors = [
        camera
    ],
)

```

1.1 EKF SLAM with Known Correspondences

First we explore the algorithm with known correspondences, this means that the landmarks are uniquely and unambiguously identifiable for every measurement.

The implementation of the algorithm serves as base to then extend it in the case where the correspondence can not be unambiguously deduced from the measurement.

The reference algorithm uses as state variables for the estimation the following definition:

$$\mu = [\mu_x \quad \mu_m]^T = [\mu_x \quad \mu_m^{(0)} \quad \dots \quad \mu_m^{(N-1)}]^T$$

where $\mu_x = (\hat{x}, \hat{y}, \hat{\theta})$ is the position estimate for the robot's pose and $\mu_m^{(k)} = (\hat{x}_{m,k}, \hat{y}_{m,k})$ is the position estimate for the k-th landmark and for $k \in \{0, \dots, N-1\}$. Similarly the covariances are defined and can be sliced for computation as:

$$\Sigma = \begin{bmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{bmatrix} = \begin{bmatrix} \Sigma_{xx} & \Sigma_{x,m_0} & \dots & \Sigma_{x,m_{N-1}} \\ \Sigma_{m_0,x} & \Sigma_{m_0,m_0} & \dots & \Sigma_{m_0,m_{N-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_{N-1},x} & \Sigma_{m_{N-1},m_0} & \dots & \Sigma_{m_{N-1},m_{N-1}} \end{bmatrix} = \Sigma^T$$

with

$$\Sigma_{xm} = [\Sigma_{x,m_0} \quad \dots \quad \Sigma_{x,m_{N-1}}]; \Sigma_{mx} = \Sigma_{xm}^T$$

and

$$\Sigma_{mm} = \begin{bmatrix} \Sigma_{m_0,m_0} & \dots & \Sigma_{m_0,m_{N-1}} \\ \vdots & \ddots & \vdots \\ \Sigma_{m_{N-1},m_0} & \dots & \Sigma_{m_{N-1},m_{N-1}} \end{bmatrix}$$

1.2 About the implementation

The algorithms equations can then be developed to understand how the update of a sigle landmark affects only a specific part of the covariace matrix Σ . This does not only help to better understand how the update step adjusts the uncertainty of the landmarks, but also allows to optimize the way the algorithm is implemented numerically. The main equations of the algorithm are:

- **Prediction Step (Prior Belief):** Propagates the robot pose estimate and its covariance through the motion model to predict the state at time t before incorporating measurements. The motion model g predicts the new mean, and the covariance update accounts for process noise.

$$\begin{aligned}\bar{\mu}_t &= g(u_t, \mu_{t-1}) \\ Q_t \bar{\Sigma}_t &= G_t \Sigma_t G_t^T + F_x^T R F_x\end{aligned}$$

- **Correction Step (Posterior Belief):** Refines the state estimate by fusing sensor measurements. The Kalman gain K_i^t weights the measurement innovation $dz_i^t = (z_i^t - \hat{z}_i^t)$, updating both the mean and covariance to produce the posterior distribution after measurement incorporation.

$$\begin{aligned}K_i^t &= \bar{\Sigma}_t H_i^T (H_i \bar{\Sigma}_t H_i^T + Q_t)^{-1} \\ \mu_t &= \bar{\mu}_t + K_i^t (z_i^t - \hat{z}_i^t) \\ \Sigma_t &= (I - K_i^t H_i) \bar{\Sigma}_t\end{aligned}$$

1.2.1 Prediction Step:

In the prediction step we notice that the selector matrix F_x , used to construc G_t and add the process noise R , causes only μ_x , Σ_{xx} and Σ_{mx} to be updated. So this step basically can be reduced to the following computation:

$$\begin{aligned}\bar{\mu}_x^t &= g(u_t, \mu_{t-1}) \\ \bar{\Sigma}_{xx}^t &= J_t(\mu_x) \Sigma_{xx}^t J_t(\mu_x)^T + R \\ \bar{\Sigma}_{mx}^t &= \Sigma_{mx}^t J_t(\mu_x)^T\end{aligned}$$

Note on Σ_{mx} that needs to be updated, this requires as well Σ_{xm} to be updated, however, by symmetry we know that $\Sigma_{mx} = \Sigma_{xm}^T$

In the actual code we can implement this by slicing the Σ matrix and updating only the respective regions, so that the updates are incorporated automatically at the right position.

1.2.2 Correction Step:

The correction step needs a bit more manipulations but can be rewritten as follows:

The innovation: Let the innovation Ψ be defined as follows:

$$\Psi = H_i \bar{\Sigma}_t H_i^T + Q_t$$

The matrix H_i is here constructed with a selector matrix $F_{x,i}$ from the measuremnt model's jacobian H:

$$H_i = HF_i = [H_x \quad H_m] \begin{bmatrix} I_3 & 0 & \dots & \dots & \dots & 0 \\ 0 & 0 & \dots & I_2 & \dots & 0 \end{bmatrix} = [H_x \quad 0 \quad \dots \quad H_m \quad \dots \quad 0]$$

with H_x representing the Jacobian of the measurement function h evaluated with respect to the robot's state vector $[x, y, \theta]^T$ and H_m , the Jacobian of the measurement function evaluated with respect to the specific landmark's state vector $[x_m, y_m, s]^T$ in the global map, defined as follows.

$$H_x = \begin{bmatrix} -\frac{dx}{\sqrt{q}} & -\frac{dy}{\sqrt{q}} & 0 \\ \frac{dy}{q} & -\frac{dx}{q} & -1 \\ 0 & 0 & 0 \end{bmatrix} \quad H_m = \begin{bmatrix} \frac{dx}{\sqrt{q}} & \frac{dy}{\sqrt{q}} & 0 \\ -\frac{dy}{q} & \frac{dx}{q} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Note on the landmark state dimensionality. In EKF-SLAM with known correspondences, the feature's identity is already known from the correspondence, so the measurement-model component of the feature signature is not required in the state representation. Consequently, each landmark is represented only by its (x, y) coordinates, giving a 2-dimensional landmark state and covariance block.

In EKF-SLAM with unknown correspondences, the full feature signature must be retained to allow measurements to be matched against existing landmarks. In that case, the landmark state is 3-dimensional. This explains the difference between the 2-dimensional landmark representation used here and the 3-dimensional representation required in the unknown-correspondence case.

Knowing H_i 's shape we can then expand the expression to compute Ψ :

$$H_i \bar{\Sigma}_t H_i^T = [H_x \quad 0 \quad \dots \quad H_m \quad \dots \quad 0] \begin{bmatrix} \Sigma_{xx} & \Sigma_{x,m_0} & \dots & \Sigma_{x,m_i} & \dots & \Sigma_{x,m_{N-1}} \\ \Sigma_{m_0,x} & \Sigma_{m_0,m_0} & \dots & \Sigma_{m_0,m_i} & \dots & \Sigma_{m_0,m_{N-1}} \\ \vdots & \vdots & \ddots & \vdots & & \vdots \\ \Sigma_{m_i,x} & \Sigma_{m_i,m_0} & \dots & \Sigma_{m_i,m_i} & \dots & \Sigma_{m_i,m_{N-1}} \\ \vdots & \vdots & & \vdots & \ddots & \vdots \\ \Sigma_{m_{N-1},x} & \Sigma_{m_{N-1},m_0} & \dots & \Sigma_{m_{N-1},m_i} & \dots & \Sigma_{m_{N-1},m_{N-1}} \end{bmatrix} [H_x \quad 0 \quad \dots \quad H_m \quad \dots \quad 0]^T$$

We can then rewrite these expressions to simplify the notation as follows:

$$H_i \bar{\Sigma}_t H_i^T = [A_x \quad A_{m_0} \quad \dots \quad A_{m_i} \quad \dots \quad A_{m_{N-1}}] [H_x \quad 0 \quad \dots \quad H_m \quad \dots \quad 0]^T$$

With $A_x = H_x \Sigma_{xx} + H_m \Sigma_{m_i,x}$ and $A_{m_k} = H_x \Sigma_{x,m_k} + H_m \Sigma_{m_i,m_k}$ for $k \in \{0, \dots, N-1\}$, which yields the final expression for Ψ :

$$\Psi = A_x H_x^T + A_{m_i} H_m^T + Q_t = \dots = H_x \Sigma_{xx} H_x^T + H_m \Sigma_{m_i,x} H_x^T + H_x \Sigma_{x,m_i} H_m^T + H_m \Sigma_{m_i,m_i} H_m^T + Q_t$$

The Kalman Gain Similarly, we can compute the product $\bar{\Sigma}_t H_i^T$ to get to an expression for K_i

$$K_i = [K_x^{(i)} \quad K_0^{(i)} \quad \dots \quad K_i^{(i)} \quad \dots \quad K_{N-1}^{(i)}]^T$$

where $K_x^{(i)} = (\Sigma_{xx} H_x^T + \Sigma_{x,m_i} H_m^T) \Psi^{-1}$ and $K_k^{(i)} = (\Sigma_{m_k x} H_x^T + \Sigma_{m_k m_i} H_m^T) \Psi^{-1}$

This allows us to compute the final piece of the puzzle which is the matrix $I - K_i^t H_i$. If we compute the product $K_i^t H_i$ it is easy to see that only a limited set of columns are touched by the update as:

$$K_i^t H_i = \begin{bmatrix} K_x^{(i)} H_x & 0 & \dots & K_i^{(i)} H_m & \dots & 0 \\ K_0^{(i)} H_x & 0 & \dots & K_0^{(i)} H_m & \dots & 0 \\ \vdots & \vdots & & \vdots & & \vdots \\ K_{N-1}^{(i)} H_x & 0 & \dots & K_{N-1}^{(i)} H_m & \dots & 0 \end{bmatrix}^T$$

These observations allow the implementation to operate directly on the relevant blocks of the state and covariance matrices rather than performing full-matrix multiplications involving potentially hundreds of rows and columns.

The resulting implementation is both more computationally efficient and more representative of the underlying structure of EKF-SLAM: although the state contains all landmarks, a measurement directly involves only the robot pose and the observed landmark, while the existing cross-correlations propagate the update to the rest of the map.

```
[ ]: ekf = EKFSlamKnownCorrespondences(n_landmarks)

[ ]: # trajectory
def time2n_frames(t, dt):
    return int(np.round(t / dt))

trajectory = [
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, 0.,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, -np.pi / 5,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, 0.,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, -np.pi / 5,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, 0.,
    ↪dt),
]

[ ]: n_frames = len(trajectory)

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(12, 12),
    ↪layout="tight")
fig.suptitle("EKF with Known Correspondences")

#update function for animation
sampling_intervall = 1 / measurement_frequency #[s]
time_since_last_measurement = 0 #[s]
```

```

applied_controls      = []
position_trajectory   = [x0.as_array.copy()]
mu_x_trajectory       = [ekf.mu_x.copy()]
sigma_x_trajectory    = [ekf.sigma_x.copy()]
lm_idx = 0
mu_lm_i_trajectory    = [ekf.get_mu_m_i(lm_idx).copy()]
sigma_lm_i_trajectory = [ekf.get_sigma_m_i(lm_idx).copy()]

def update(frame):
    global applied_controls, sampling_intervall, time_since_last_measurement, \
           position_trajectory, mu_x_trajectory, mu_lm_i_trajectory, \
           sigma_lm_i_trajectory, sigma_x_trajectory

    ax1.clear()
    ax2.clear()
    ax3.clear()
    ax4.clear()

    ax1.set_title("Robot in Ground Truth")
    ax1.set_xlabel("$x$ [m]")
    ax1.set_ylabel("$y$ [m]")

    ax2.set_title("Robot's Perception")
    ax2.set_xlabel("$x$ [m]")
    ax2.set_ylabel("$y$ [m]")

    ax3.set_title("EKF Estimates")
    ax3.set_xlabel("$x$ [m]")
    ax3.set_ylabel("$y$ [m]")

    ax4.set_title("EKF on GT Overlay")
    ax4.set_xlabel("$x$ [m]")
    ax4.set_ylabel("$y$ [m]")

    # Apply control and store effectively applied noisy input
    u = trajectory[frame]
    u_eff = robot.applyControl(u, motion_noise=True)
    applied_controls.append(u_eff)

    # Take a measurement at given frequency
    if time_since_last_measurement >= sampling_intervall:
        time_since_last_measurement = 0
        robot.measure(env, detection_noise=True)
    else:
        time_since_last_measurement += dt

    # Update EKF

```

```

redaings = robot.getCurrentSensorReadings()[0]
detected_features = None
correspondences = None

if redaings is not None:
    detected_features = redaings['features']
    correspondences = redaings['correspondences']

ekf.update(u, detected_features)

position_trajectory.append(robot.pos.as_array.copy())
mu_x_trajectory.append(ekf.mu_x.copy())
sigma_x_trajectory.append(ekf.sigma_x.copy())
mu_lm_i_trajectory.append(ekf.get_mu_m_i(lm_idx).copy())
sigma_lm_i_trajectory.append(ekf.get_sigma_m_i(lm_idx).copy())

# AX 1: Robot in Environment
env.draw(ax1)
robot.draw(ax1, r = 0.5, draw_sensors=False)

# AX 2: Robot's Perception
if correspondences: env.draw(ax2, correspondences=correspondences,
↪set_limits=False)
robot.draw(ax2, r = 0.5, draw_sensors=True)
ekf.draw(ax2, env.colors)
ax2.set_xlim(robot.pos.x - (camera.range + 0.5), robot.pos.x + (camera.
↪range + 0.5))
ax2.set_ylim(robot.pos.y - (camera.range + 0.5), robot.pos.y + (camera.
↪range + 0.5))
ax2.legend().remove()

# AX 3: EKF Estimates
ekf.draw(ax3, env.colors)
ax3.set_xlim(x_lims[0], x_lims[1])
ax3.set_ylim(y_lims[0], y_lims[1])

# AX 4: EKF on GT Overlay
ekf.draw(ax4, env.colors)
env.draw(ax4)
ax4.legend().remove()

print(f"Progress: {(frame / (n_frames - 1)) * 100}%.2f%%", end="\r")

ani = FuncAnimation(fig, update, frames=np.arange(0, n_frames), interval=dt *
↪1000)

gif_path = 'img/sim_EKF_SLAM_KC.gif'

```

```
ani.save(gif_path, writer='pillow', fps=20)

plt.close(fig)
```

```
[ ]: display(Image(filename=gif_path))
```

```
<IPython.core.display.Image object>
```

```
[ ]: v_traj= []
w_traj = []
v_eff_traj = []
w_eff_traj = []
t_traj = []

t = 0
for u, u_eff in zip(trajjectory, applied_controls):
    t_traj.append(t)
    v_traj.append(u.v)
    w_traj.append(u.w)
    v_eff_traj.append(u_eff.v)
    w_eff_traj.append(u_eff.w)
    t += dt

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))
ax1.set_title("Trajectory for $v$")
ax1.set_xlabel("Time [s]")
ax1.set_ylabel(r"$v$ [m/s]")

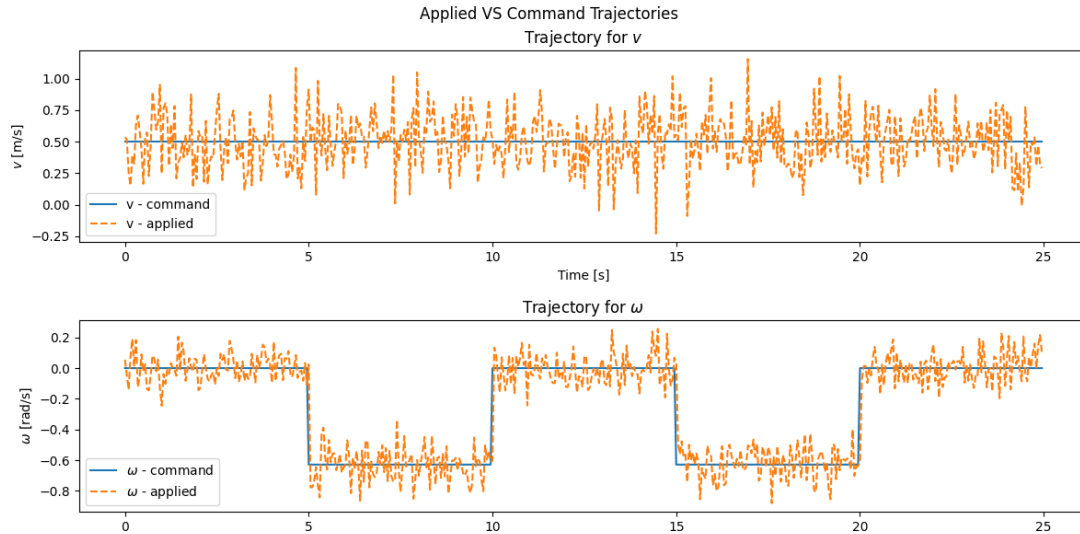
ax2.set_title("Trajectory for $\omega$")
ax2.set_ylabel(r"$\omega$ [rad/s]")

ax1.plot(t_traj, v_traj, label="v - command")
ax1.plot(t_traj, v_eff_traj, linestyle='--', label="v - applied")
ax1.legend()

ax2.plot(t_traj, w_traj, label="$\omega$ - command")
ax2.plot(t_traj, w_eff_traj, linestyle='--', label="$\omega$ - applied")
ax2.legend()

fig.suptitle("Applied VS Command Trajectories")

fig.tight_layout()
fig.subplots_adjust(top=0.9)
```



```
[ ]: # Unpack trajectories
x_r, y_r, th_r = [], [], []
x_ekf, y_ekf, th_ekf = [], [], []
x_lm, y_lm = [], []

x_ekf_std, y_ekf_std, th_ekf_std = [], [], []
x_lm_std, y_lm_std = [], []

for pos, mu_x, sigma_x, mu_lm, sigma_lm in zip(
    position_trajectory,
    mu_x_trajectory,
    sigma_x_trajectory,
    mu_lm_i_trajectory,
    sigma_lm_i_trajectory,
):
    # Means
    x_r_i, y_r_i, th_r_i = pos.squeeze()
    x_ekf_i, y_ekf_i, th_ekf_i = mu_x.squeeze()
    x_lm_i, y_lm_i = mu_lm.squeeze()

    x_r.append(x_r_i)
    y_r.append(y_r_i)
    th_r.append(th_r_i)

    x_ekf.append(x_ekf_i)
    y_ekf.append(y_ekf_i)
    th_ekf.append(th_ekf_i)
```

```

x_lm.append(x_lm_i)
y_lm.append(y_lm_i)

# Marginal standard deviations = sqrt(diagonal of covariance)
x_ekf_std.append(np.sqrt(sigma_x[0, 0]))
y_ekf_std.append(np.sqrt(sigma_x[1, 1]))
th_ekf_std.append(np.sqrt(sigma_x[2, 2]))

x_lm_std.append(np.sqrt(sigma_lm[0, 0]))
y_lm_std.append(np.sqrt(sigma_lm[1, 1]))

# Convert to arrays
x_r, y_r, th_r = map(np.asarray, (x_r, y_r, th_r))
x_ekf, y_ekf, th_ekf = map(np.asarray, (x_ekf, y_ekf, th_ekf))
x_lm, y_lm = map(np.asarray, (x_lm, y_lm))

x_ekf_std, y_ekf_std, th_ekf_std = map(
    np.asarray, (x_ekf_std, y_ekf_std, th_ekf_std)
)
x_lm_std, y_lm_std = map(np.asarray, (x_lm_std, y_lm_std))

```

```

[ ]: t = np.arange(len(x_ekf)) * dt

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 9))

ax1.plot(t, x_r, label="Ground truth x", color="#2889e4")
ax1.plot(t, x_ekf, label="EKF x", color="#d6591f", linestyle='--')
ax1.fill_between(t, x_ekf - x_ekf_std, x_ekf + x_ekf_std, alpha=0.2,
    color="#fdb96c",)
ax1.legend()

ax2.plot(t, y_r, label="Ground truth y", color="#2889e4")
ax2.plot(t, y_ekf, label="EKF y", linestyle='--', color="#d6591f")
ax2.fill_between(t, y_ekf - y_ekf_std, y_ekf + y_ekf_std, alpha=0.2,
    color="#fdb96c")
ax2.legend()

ax3.plot(t, th_r, label="Ground truth ", color="#2889e4")
ax3.plot(t, th_ekf, label="EKF ", linestyle="--", color="#d6591f")
ax3.fill_between(t, th_ekf - th_ekf_std, th_ekf + th_ekf_std, alpha=0.2,
    color="#fdb96c")
ax3.legend()

ax1.set_ylabel("x position [m]")
ax2.set_ylabel("y position [m]")
ax3.set_ylabel("Orientation [rad]")
ax3.set_xlabel("Time [s]")

```

```

ax1.set_title("Robot x-position")
ax2.set_title("Robot y-position")
ax3.set_title("Robot orientation")

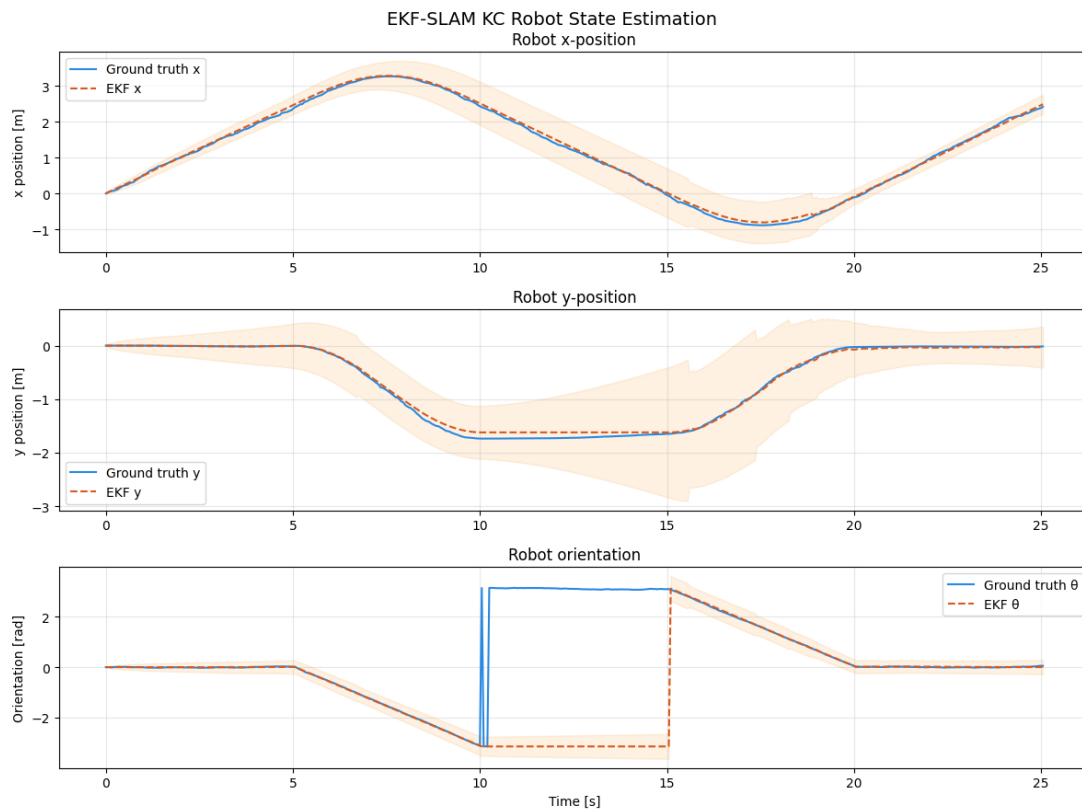
ax1.grid(True, alpha=0.3)
ax2.grid(True, alpha=0.3)
ax3.grid(True, alpha=0.3)

fig.suptitle("EKF-SLAM KC Robot State Estimation", fontsize=14)

fig.tight_layout()
fig.subplots_adjust(top=0.93)

plt.show()

```



```

[ ]: fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))

c_idx = next((k for k, v in ekf.correspondence.items() if v == lm_idx), None)

```

```

ax1.axhline(y=env.landmarks[c_idx].pos.x, label="Landmark x", color=env.
    ↪colors[c_idx])
ax1.plot(t, x_lm, label="Landmark EKF x", linestyle='--', color=env.
    ↪colors[c_idx])
ax1.fill_between(t, x_lm - x_lm_std, x_lm + x_lm_std, alpha=0.2, color=env.
    ↪colors[c_idx])
ax1.legend()

ax2.axhline(y=env.landmarks[c_idx].pos.y, label="Landmark y", color=env.
    ↪colors[c_idx])
ax2.plot(t, y_lm, label="Landmark EKF y", linestyle='--', color=env.
    ↪colors[c_idx])
ax2.fill_between(t, y_lm - y_lm_std, y_lm + y_lm_std, alpha=0.2, color=env.
    ↪colors[c_idx])
ax2.legend()

ax1.set_ylabel("x position [m]")
ax2.set_ylabel("y position [m]")
ax2.set_xlabel("Time [s]")

ax1.set_title("Landmark x-position")
ax2.set_title("Landmark y-position")

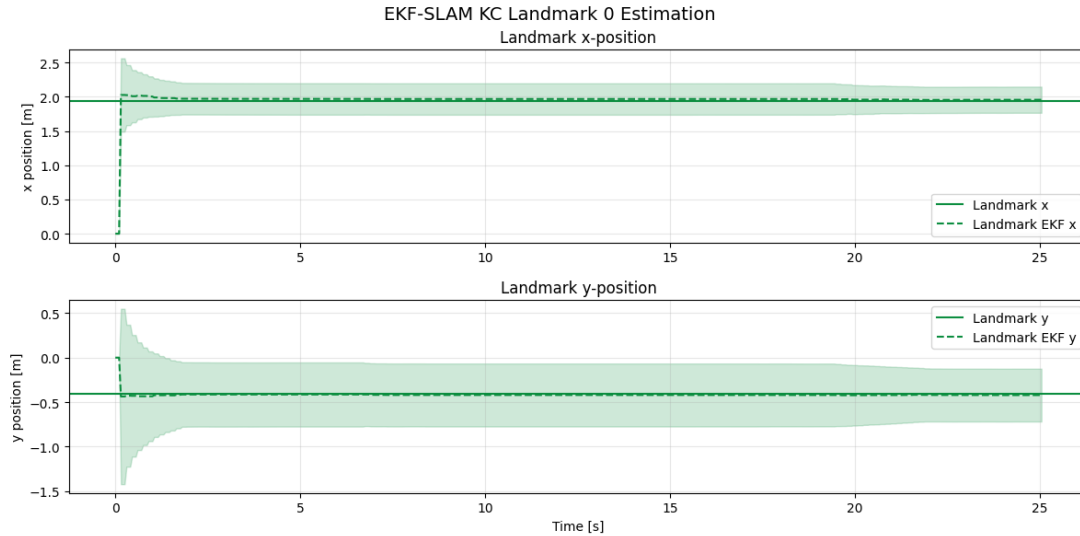
ax1.grid(True, alpha=0.3)
ax2.grid(True, alpha=0.3)

fig.suptitle(f"EKF-SLAM KC Landmark {lm_idx} Estimation", fontsize=14)

fig.tight_layout()
fig.subplots_adjust(top=0.90)

plt.show()

```



1.3 EKF SLAM with Unknown Correspondences

The algorithm differs only slightly from the EKF SLAM with Known Correspondences. The major difference lies in the correction step. Indeed the prediction step remains exactly the same.

Since the correspondence can not be setup, the state of the landmark estimate must now include the signature. When a new measurement is processed, now the measurement is compared against all stored landmark estimates by means of the malahanobis distance π , which is to be minimized. The measurement is then assigned to the landmark estimate that minimizes given distance π . Note that to regulate the addition of new landmarks to the constructed map of estimates, the threshold α is used: should all landmark estimates yield a distance π greater than alpha, then we consider the landmark to be a different one of the ones already detected and initialize a new estimate in the array of landmark estimates.

The mathematical foundation setup in the previous section still holds and the same optimizations can be applied, by computing the new estimate and covariances by means of slices only of the previous ones. This is visible in the code by the inheritance pattern:

```
class EKFSlamBase:
    # Implements slicing and position update

class EKFSlamKnownCorrespondences(EKFSlamBase):
    # ...

class EKFSlamUnknownCorrespondences(EKFSlamBase):
    # ...
```

```
[ ]: #define the robot
robot = Robot(
    x0,
```

```

    sensors = [
        camera
    ],
)

```

```
[ ]: ekf = EKFSlamUnknownCorrespondences(n_landmarks)
```

```
[ ]: # trajectory
def time2n_frames(t, dt):
    return int(np.round(t / dt))

trajectory = [
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, 0.          ,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, -np.pi / 5,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, 0.          ,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, -np.pi / 5,
    ↪dt),
    *generateConstantVelocityTrajectory(time2n_frames(5, dt), 0.5, 0.          ,
    ↪dt),
]

```

```
[ ]: n_frames = len(trajectory)

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(12, 12),
    ↪layout="tight")
fig.suptitle("EKF with Unknown Correspondences")

#update function for animation
sampling_intervall = 1 / measurement_frequency #[s]
time_since_last_measurement = 0 #[s]

applied_controls      = []
position_trajectory   = [x0.as_array.copy()]
mu_x_trajectory       = [ekf.mu_x.copy()]
sigma_x_trajectory    = [ekf.sigma_x.copy()]
lm_idx = 0
mu_lm_i_trajectory    = [ekf.get_mu_m_i(lm_idx).copy()]
sigma_lm_i_trajectory  = [ekf.get_sigma_m_i(lm_idx).copy()]

def update(frame):
    global applied_controls, sampling_intervall, time_since_last_measurement, \
        position_trajectory, mu_x_trajectory, mu_lm_i_trajectory, \
        sigma_lm_i_trajectory, sigma_x_trajectory

```

```

ax1.clear()
ax2.clear()
ax3.clear()
ax4.clear()

ax1.set_title("Robot in Ground Truth")
ax1.set_xlabel("$x$ [m]")
ax1.set_ylabel("$y$ [m]")

ax2.set_title("Robot's Perception")
ax2.set_xlabel("$x$ [m]")
ax2.set_ylabel("$y$ [m]")

ax3.set_title("EKF Estimates")
ax3.set_xlabel("$x$ [m]")
ax3.set_ylabel("$y$ [m]")

ax4.set_title("EKF on GT Overlay")
ax4.set_xlabel("$x$ [m]")
ax4.set_ylabel("$y$ [m]")

# Apply control and store effectively applied noisy input
u = trajectory[frame]
u_eff = robot.applyControl(u, motion_noise=True)
applied_controls.append(u_eff)

# Take a measurement at given frequency
if time_since_last_measurement >= sampling_intervall:
    time_since_last_measurement = 0
    robot.measure(env,detection_noise=True)
else:
    time_since_last_measurement += dt

# Update EKF
redaings = robot.getCurrentSensorReadings()[0]
detected_features = None
correspondences = None

if redaings is not None:
    detected_features = redaings['features']
    correspondences = redaings['correspondences']

ekf.update(u, detected_features)

position_trajectory.append(robot.pos.as_array.copy())
mu_x_trajectory.append(ekf.mu_x.copy())

```

```

sigma_x_trajectory.append(ekf.sigma_x.copy())
mu_lm_i_trajectory.append(ekf.get_mu_m_i(lm_idx).copy())
sigma_lm_i_trajectory.append(ekf.get_sigma_m_i(lm_idx).copy())

# AX 1: Robot in Environment
env.draw(ax1)
robot.draw(ax1, r = 0.5, draw_sensors=False)

# AX 2: Robot's Perception
if correspondences: env.draw(ax2, correspondences=correspondences,
↪set_limits=False)
robot.draw(ax2, r = 0.5, draw_sensors=True)
ekf.draw(ax2, env.colors)
ax2.set_xlim(robot.pos.x - (camera.range + 0.5), robot.pos.x + (camera.
↪range + 0.5))
ax2.set_ylim(robot.pos.y - (camera.range + 0.5), robot.pos.y + (camera.
↪range + 0.5))
ax2.legend().remove()

# AX 3: EKF Estimates
ekf.draw(ax3, env.colors)
ax3.set_xlim(x_lims[0], x_lims[1])
ax3.set_ylim(y_lims[0], y_lims[1])

# AX 4: EKF on GT Overlay
ekf.draw(ax4, env.colors)
env.draw(ax4)
ax4.legend().remove()

print(f"Progress: {((frame / (n_frames - 1)) * 100):.2f}%", end="\r")

ani = FuncAnimation(fig, update, frames=np.arange(0, n_frames), interval=dt *
↪1000)

gif_path = 'img/sim_EKF_SLAM_UC.gif'
ani.save(gif_path, writer='pillow', fps=20)

plt.close(fig)

```

```
[ ]: display(Image(filename=gif_path))
```

<IPython.core.display.Image object>

1.4 About α

Alpha is the threshold that influences the acceptance of new landmarks. High values of alpha tend to reduce the numbers of newly added landmarks, while lower numbers tend to increase the

number of new landmarks. Indeed, if too low, several landmark estimates are created for the same landmark in the GT, conversly, too high alphas map all measurements to a reduced and incomplete set of landmark estimates. This value is empirical, for the 2D case a range of 3-6 has proven to work relatively well.

```
[ ]: v_traj= []
      w_traj = []
      v_eff_traj = []
      w_eff_traj = []
      t_traj = []

      t = 0
      for u, u_eff in zip(trajjectory, applied_controls):
          t_traj.append(t)
          v_traj.append(u.v)
          w_traj.append(u.w)
          v_eff_traj.append(u_eff.v)
          w_eff_traj.append(u_eff.w)
          t += dt

      fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))
      ax1.set_title("Trajectory for $v$")
      ax1.set_xlabel("Time [s]")
      ax1.set_ylabel(r"$v$ [m/s]")

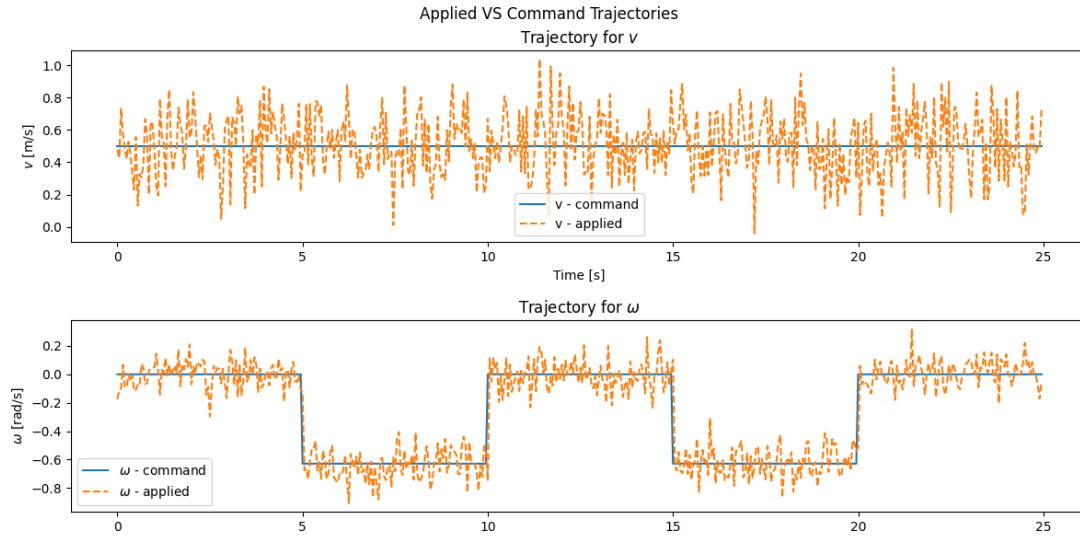
      ax2.set_title("Trajectory for $\omega$")
      ax2.set_ylabel(r"$\omega$ [rad/s]")

      ax1.plot(t_traj, v_traj, label="v - command")
      ax1.plot(t_traj, v_eff_traj, linestyle='--', label="v - applied")
      ax1.legend()

      ax2.plot(t_traj, w_traj, label="$\omega$ - command")
      ax2.plot(t_traj, w_eff_traj, linestyle='--', label="$\omega$ - applied")
      ax2.legend()

      fig.suptitle("Applied VS Command Trajectories")

      fig.tight_layout()
      fig.subplots_adjust(top=0.9)
```



```
[ ]: # Unpack trajectories
x_r, y_r, th_r = [], [], []
x_ekf, y_ekf, th_ekf = [], [], []
x_lm, y_lm = [], []

x_ekf_std, y_ekf_std, th_ekf_std = [], [], []
x_lm_std, y_lm_std = [], []

for pos, mu_x, sigma_x, mu_lm, sigma_lm in zip(
    position_trajectory,
    mu_x_trajectory,
    sigma_x_trajectory,
    mu_lm_i_trajectory,
    sigma_lm_i_trajectory,
):
    # Means
    x_r_i, y_r_i, th_r_i = pos.squeeze()
    x_ekf_i, y_ekf_i, th_ekf_i = mu_x.squeeze()
    x_lm_i, y_lm_i, _ = mu_lm.squeeze()

    x_r.append(x_r_i)
    y_r.append(y_r_i)
    th_r.append(th_r_i)

    x_ekf.append(x_ekf_i)
    y_ekf.append(y_ekf_i)
    th_ekf.append(th_ekf_i)
```

```

x_lm.append(x_lm_i)
y_lm.append(y_lm_i)

# Marginal standard deviations = sqrt(diagonal of covariance)
x_ekf_std.append(np.sqrt(sigma_x[0, 0]))
y_ekf_std.append(np.sqrt(sigma_x[1, 1]))
th_ekf_std.append(np.sqrt(sigma_x[2, 2]))

x_lm_std.append(np.sqrt(sigma_lm[0, 0]))
y_lm_std.append(np.sqrt(sigma_lm[1, 1]))

# Convert to arrays
x_r, y_r, th_r = map(np.asarray, (x_r, y_r, th_r))
x_ekf, y_ekf, th_ekf = map(np.asarray, (x_ekf, y_ekf, th_ekf))
x_lm, y_lm = map(np.asarray, (x_lm, y_lm))

x_ekf_std, y_ekf_std, th_ekf_std = map(
    np.asarray, (x_ekf_std, y_ekf_std, th_ekf_std)
)
x_lm_std, y_lm_std = map(np.asarray, (x_lm_std, y_lm_std))

```

```

[ ]: t = np.arange(len(x_ekf)) * dt

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 9))

ax1.plot(t, x_r, label="Ground truth x", color="#2889e4")
ax1.plot(t, x_ekf, label="EKF x", color="#d6591f", linestyle='--')
ax1.fill_between(t, x_ekf - x_ekf_std, x_ekf + x_ekf_std, alpha=0.2,
    color="#fdb96c",)
ax1.legend()

ax2.plot(t, y_r, label="Ground truth y", color="#2889e4")
ax2.plot(t, y_ekf, label="EKF y", linestyle='--', color="#d6591f")
ax2.fill_between(t, y_ekf - y_ekf_std, y_ekf + y_ekf_std, alpha=0.2,
    color="#fdb96c")
ax2.legend()

ax3.plot(t, th_r, label="Ground truth ", color="#2889e4")
ax3.plot(t, th_ekf, label="EKF ", linestyle="--", color="#d6591f")
ax3.fill_between(t, th_ekf - th_ekf_std, th_ekf + th_ekf_std, alpha=0.2,
    color="#fdb96c")
ax3.legend()

ax1.set_ylabel("x position [m]")
ax2.set_ylabel("y position [m]")
ax3.set_ylabel("Orientation [rad]")
ax3.set_xlabel("Time [s]")

```

```

ax1.set_title("Robot x-position")
ax2.set_title("Robot y-position")
ax3.set_title("Robot orientation")

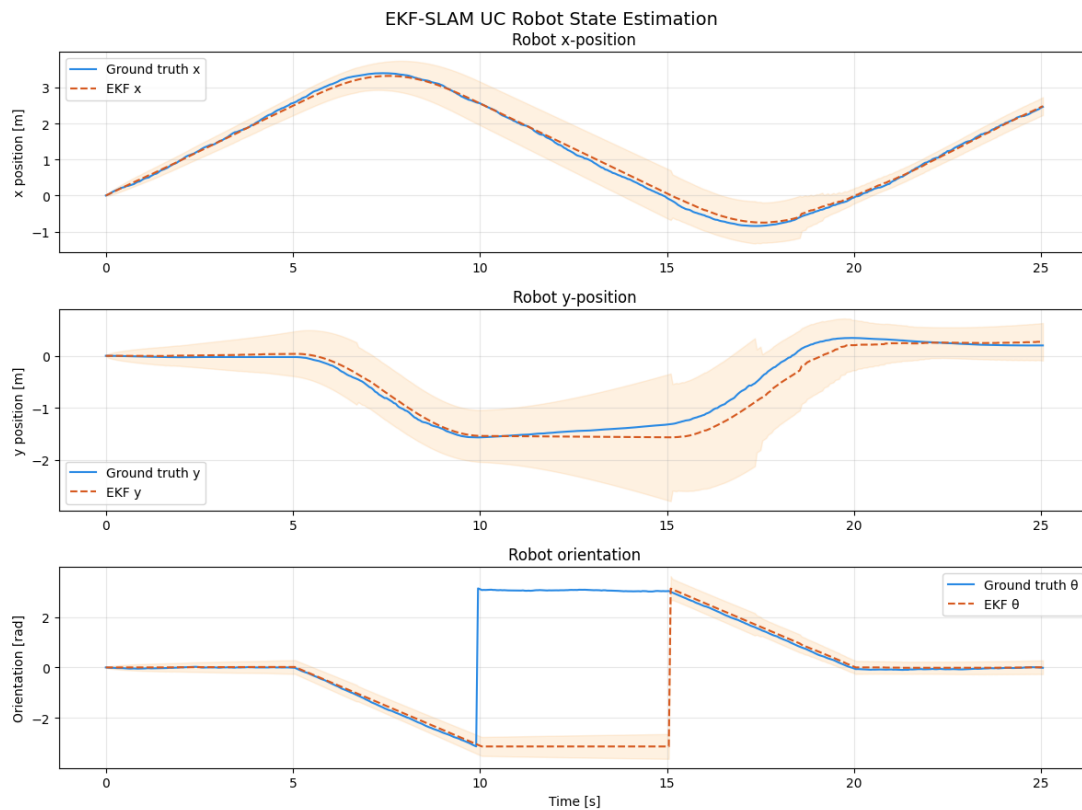
ax1.grid(True, alpha=0.3)
ax2.grid(True, alpha=0.3)
ax3.grid(True, alpha=0.3)

fig.suptitle("EKF-SLAM UC Robot State Estimation", fontsize=14)

fig.tight_layout()
fig.subplots_adjust(top=0.93)

plt.show()

```



```

[ ]: fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6))

```

```

# here we use c_idx of the EKF SLAM KC since we know that the first landmark
# that should be detected is the one with c_idx as signature, under the
# assumption, that map, sensor and initial position stay the same

```

```

ax1.axhline(y=env.landmarks[c_idx].pos.x, label="Landmark x")
ax1.plot(t, x_lm, label="Landmark EKF x", linestyle='--')
ax1.fill_between(t, x_lm - x_lm_std, x_lm + x_lm_std, alpha=0.2)
ax1.legend()

ax2.axhline(y=env.landmarks[c_idx].pos.y, label="Landmark y")
ax2.plot(t, y_lm, label="Landmark EKF y", linestyle='--')
ax2.fill_between(t, y_lm - y_lm_std, y_lm + y_lm_std, alpha=0.2)
ax2.legend()

ax1.set_ylabel("x position [m]")
ax2.set_ylabel("y position [m]")
ax2.set_xlabel("Time [s]")

ax1.set_title("Landmark x-position")
ax2.set_title("Landmark y-position")

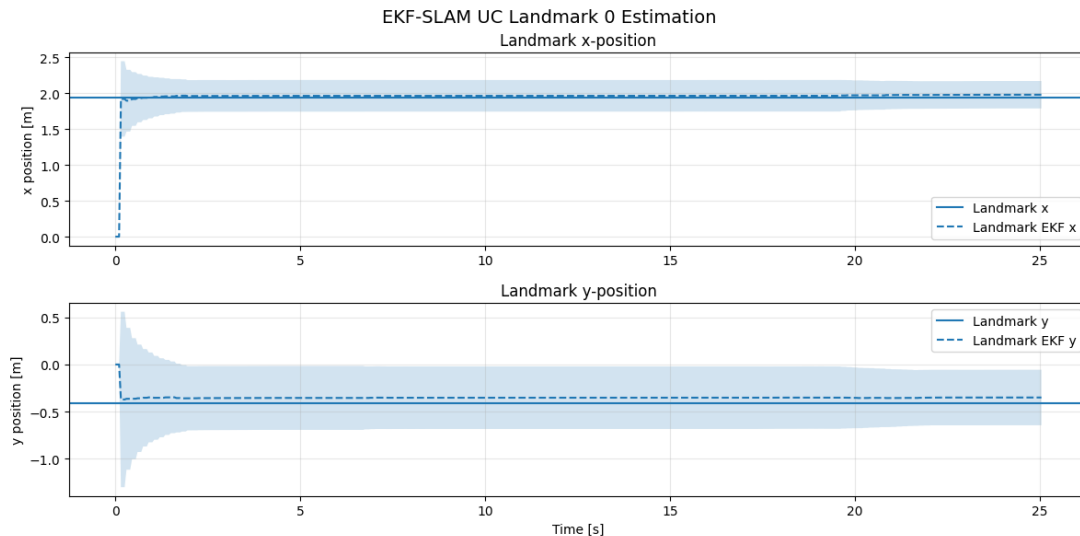
ax1.grid(True, alpha=0.3)
ax2.grid(True, alpha=0.3)

fig.suptitle(f"EKF-SLAM UC Landmark {lm_idx} Estimation", fontsize=14)

fig.tight_layout()
fig.subplots_adjust(top=0.90)

plt.show()

```



Trabalho 7

September 23, 2026

1 FastSLAM 1.0 and 2.0 — Motion Noise and Algorithm Comparison

This notebook evaluates FastSLAM 1.0 and FastSLAM 2.0 under two different motion-noise levels.

1.1 1. Imports

```
[13]: import os
import numpy as np
import matplotlib.pyplot as plt

from robotics.geometry import Pose, Angle
from robotics.maps import Landmark, LandmarkMap
from robotics.robot import Robot
from robotics.camera import Camera
from robotics.control import VelocityControl
from robotics.fastslam import FastSLAM, LandmarkEstimate
from robotics.fastslam2 import FastSLAM2

from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display
```

1.2 2. Simulation Configuration

```
[14]: dt = 0.05
N_PARTICLES = 50

FOV = Angle.from_deg(100)
# SENSOR_RANGE = 3.0
SENSOR_RANGE = 5.0

RANGE_STD = 0.05
BEARING_STD = np.deg2rad(1.0)

Q = np.diag([
    RANGE_STD**2,
    BEARING_STD**2
```

```

])

x0 = Pose(
    -3.0,
    -2.0,
    Angle.from_deg(0)
)

LOW_MOTION_NOISE = (0.20, 0.02)
HIGH_MOTION_NOISE = (0.50, 0.05)

```

1.2.1 Measurement Model

For a particle with pose

$$\mathbf{x}_t^{[k]} = \begin{bmatrix} x_t^{[k]} & y_t^{[k]} & \theta_t^{[k]} \end{bmatrix}^T,$$

and a landmark estimate

$$\mu_j = \begin{bmatrix} m_{j,x} & m_{j,y} \end{bmatrix}^T,$$

the relative displacement between the particle and the landmark is

$$\delta_j = \begin{bmatrix} \Delta x_j \\ \Delta y_j \end{bmatrix} = \begin{bmatrix} m_{j,x} - x_t^{[k]} \\ m_{j,y} - y_t^{[k]} \end{bmatrix},$$

with

$$q_j = \delta_j^T \delta_j = \Delta x_j^2 + \Delta y_j^2.$$

The predicted range-bearing measurement is therefore

$$\hat{\mathbf{z}}_j = h(\mu_j, \mathbf{x}_t^{[k]}) = \begin{bmatrix} \sqrt{q_j} \\ \text{atan2}(\Delta y_j, \Delta x_j) - \theta_t^{[k]} \end{bmatrix}.$$

The measurement noise is modeled as a zero-mean Gaussian distribution with covariance

$$\mathbf{Q}_t = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix},$$

where (σ_r) and (σ_ϕ) represent the range and bearing measurement uncertainties, respectively.

1.2.2 Measurement Jacobian and Inverse Measurement Model

The Jacobian of the measurement model with respect to the landmark position is

$$H_j = \frac{\partial h}{\partial \mu_j} = \begin{bmatrix} \frac{\Delta x_j}{\sqrt{q_j}} & \frac{\Delta y_j}{\sqrt{q_j}} \\ -\frac{\Delta y_j}{q_j} & \frac{\Delta x_j}{q_j} \end{bmatrix}.$$

For an existing landmark with covariance (Σ_j), its uncertainty projected into the measurement space is

$$S_j = H_j \Sigma_j H_j^T + Q_t.$$

This covariance is subsequently used both in the measurement likelihood and in the EKF update associated with each landmark.

When a measurement

$$\mathbf{z}_t = \begin{bmatrix} r_t \\ \phi_t \end{bmatrix}$$

is associated with a new landmark, the inverse measurement model is used to initialize its position:

$$h^{-1}(\mathbf{z}_t, \mathbf{x}_t^{[k]}) = \begin{bmatrix} x_t^{[k]} + r_t \cos(\theta_t^{[k]} + \phi_t) \\ y_t^{[k]} + r_t \sin(\theta_t^{[k]} + \phi_t) \end{bmatrix}.$$

Defining

$$\alpha = \theta_t^{[k]} + \phi_t,$$

the Jacobian of the inverse measurement model with respect to the measurement is

$$G_z = \begin{bmatrix} \cos \alpha & -r_t \sin \alpha \\ \sin \alpha & r_t \cos \alpha \end{bmatrix}.$$

Thus, the covariance of a newly initialized landmark is obtained by propagating the measurement uncertainty into Cartesian coordinates:

$$\Sigma_j = G_z Q_t G_z^T.$$

1.3 3. Landmark Map

```
[15]: landmark_positions = [
      (-1.5, -0.5),
      (1.0, -1.0),
```

```

        (3.5, -0.5),
        (4.0, 1.5),
        (1.5, 1.5),
        (-1.0, 1.5),
        (-2.0, 3.5),
        (0.5, 3.5),
        (3.0, 3.5),
        (1.0, 5.5),
        (4.0, 5.5),
    ]

    landmarks = [
        Landmark(
            Pose(x, y, Angle(0.0)),
            signature
        )
        for signature, (x, y) in enumerate(landmark_positions)
    ]

    x_lims = (-4, 7)
    y_lims = (-3, 7)

    env = LandmarkMap(
        landmarks,
        x_lims=x_lims,
        y_lims=y_lims
    )

```

1.4 4. Robot, Camera and Control Sequence

```

[16]: camera = Camera(
        Pose(0.5, 0.0, Angle.from_deg(0)),
        field_of_view=FOV,
        sensor_range=SENSOR_RANGE,
        robot_pose=x0
    )

    robot = Robot(
        x0,
        sensors=[camera]
    )

    robot_desired = Robot(x0)

```

```

[17]: commands = []

v = 0.8

```

```

turn_rate = np.pi / 2

step_distance = v * dt
horizontal_distance = 8.0
vertical_distance = 4.0

horizontal_steps = round(horizontal_distance / step_distance)
vertical_steps = round(vertical_distance / step_distance)
turn_steps = round((np.pi / 2) / (turn_rate * dt))

commands += [VelocityControl(v, 0.0, dt)] * horizontal_steps
commands += [VelocityControl(0.0, turn_rate, dt)] * turn_steps
commands += [VelocityControl(v, 0.0, dt)] * vertical_steps
commands += [VelocityControl(0.0, turn_rate, dt)] * turn_steps

commands += [VelocityControl(v, 0.0, dt)] * horizontal_steps
commands += [VelocityControl(0.0, -turn_rate, dt)] * turn_steps
commands += [VelocityControl(v, 0.0, dt)] * vertical_steps
commands += [VelocityControl(0.0, -turn_rate, dt)] * turn_steps

commands += [VelocityControl(v, 0.0, dt)] * horizontal_steps
control_sequence = commands

```

1.4.1 FastSLAM 1.0 Overview

FastSLAM represents the posterior distribution using a Rao-Blackwellized particle filter. Each particle contains a hypothesis for the robot trajectory together with an independent EKF estimate for each landmark:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(x_{1:t} \mid z_{1:t}, u_{1:t}) \prod_j p(m_j \mid x_{1:t}, z_{1:t}).$$

At each time step, the implementation performs the following operations:

1. A new robot pose is sampled for each particle according to the motion model.
2. The expected measurement and likelihood are evaluated for the landmarks contained in each particle.
3. Measurements are associated with existing landmarks or used to initialize new landmarks.
4. Associated landmarks are updated using an EKF correction.
5. A weight is assigned to each particle according to the measurement likelihoods.
6. Particles are resampled according to their normalized weights.

Consequently, each particle represents both a possible robot trajectory and its corresponding landmark map.

1.5 5. FastSLAM Simulation

The robot follows the same ground-truth trajectory in both experiments. Only the motion-noise parameters used by FastSLAM are changed.

```

[18]: def weighted_pose(particles, weights):
    x = sum(w * p.pose.x for p, w in zip(particles, weights))
    y = sum(w * p.pose.y for p, w in zip(particles, weights))

    sin_theta = sum(w * p.pose.th.sin for p, w in zip(particles, weights))
    cos_theta = sum(w * p.pose.th.cos for p, w in zip(particles, weights))

    return Pose(
        x,
        y,
        Angle.atan2(sin_theta, cos_theta)
    )

def initialize_fastslam(algorithm_cls=FastSLAM):
    slam = algorithm_cls(
        n_particles=N_PARTICLES,
        q_mat=Q
    )
    slam.initialize(x0)

    initial_cov = np.diag([0.01**2, 0.01**2])

    for particle in slam.particles:
        for landmark_id, landmark in enumerate(landmarks):
            particle.landmarks[landmark_id] = LandmarkEstimate(
                mu=np.array([[landmark.pos.x],
                             [landmark.pos.y]]),
                sigma=initial_cov.copy()
            )

    return slam

def position_error(experiment):
    gt = np.array([
        p.x, p.y
        for p in experiment["ground_truth"][1:]
    ])

    est = np.array([
        p.x, p.y
        for p in experiment["estimated"]
    ])

    return np.linalg.norm(est - gt, axis=1)

```

```
[ ]: def run_experiment(motion_noise, seed, algorithm_cls=FastSLAM):
    np.random.seed(seed)

    robot_desired.pos = Pose(x0.x, x0.y, x0.th)

    # Reset robot state.
    robot.pos = Pose(x0.x, x0.y, x0.th)
    fastslam = initialize_fastslam(algorithm_cls)

    ground_truth_poses = [
        Pose(x0.x, x0.y, x0.th)
    ]

    desired_poses = [
        Pose(x0.x, x0.y, x0.th)
    ]

    estimated_poses = []
    particle_poses = []
    effective_particle_numbers = []
    measurements_history = []
    particle_maps = []

    for control in control_sequence:

        applied_control = robot.applyControl(
            control,
            motion_noise=True
        )
        # Ground-truth robot motion is deterministic.
        _ = robot_desired.applyControl(
            control,
            motion_noise=False
        )

        # Generate the sensor observation.
        robot.measure(env)
        measurement = robot.getCurrentSensorReadings()[0]
        detected_features = measurement["features"]

        # FastSLAM prediction + measurement update.
        diagnostics = fastslam.update(
            applied_control,
            detected_features,
            motion_noise=True,
            std_mot_noise=motion_noise,
            resample=False
        )
```

```

    )

    weights = diagnostics["normalized_weights"]

    # Store particle poses before resampling.
    particle_poses.append([
        Pose(
            particle.pose.x,
            particle.pose.y,
            particle.pose.th
        )
        for particle in fastslam.particles
    ])

    estimated_poses.append(weighted_pose(fastslam.particles, weights))
    effective_particle_numbers.
    ↪append(diagnostics["effective_particle_number"]) # how many particles are
    ↪effective in representing the posterior distribution
    measurements_history.append(detected_features)

    ground_truth_poses.append(Pose(robot.pos.x, robot.pos.y, robot.pos.th))
    desired_poses.append(Pose(robot_desired.pos.x, robot_desired.pos.y,
    ↪robot_desired.pos.th))

    best_particle = max(
        fastslam.particles,
        key=lambda particle: particle.weight
    )

    particle_maps.append(
        {
            landmark_id: {
                "mu": landmark.mu.copy(),
                "sigma": landmark.sigma.copy(),
                "seen_count": landmark.seen_count,
            }
            for landmark_id, landmark
            in best_particle.landmarks.items()
            if landmark.seen_count > 1
        }
    )

    fastslam.resample()

    return {

```

```

    "desired_poses": desired_poses,
    "ground_truth": ground_truth_poses,
    "estimated": estimated_poses,
    "particles": particle_poses,
    "n_eff": np.asarray(effective_particle_numbers),
    "measurements": measurements_history,
    "particle_maps": particle_maps,
}

```

1.5.1 Particle Weighting in the Implementation

The particle-weight computation adopted in this implementation differs slightly from the FastSLAM 1.0 pseudocode considered in this work. A relevant difference between the FastSLAM 1.0 pseudocode and our implementation concerns the computation of the particle weight.

For a given particle (k) and a single observation (z_t), the pseudocode evaluates the measurement likelihood for each candidate landmark (j):

$$w_j = \frac{1}{\sqrt{|2\pi Q_j|}} \exp \left[-\frac{1}{2} (z_t - \hat{z}_j)^T Q_j^{-1} (z_t - \hat{z}_j) \right].$$

The most likely correspondence is then selected as

$$\hat{c} = \arg \max_j w_j,$$

and the particle weight is defined as

$$w^{[k]} = \max_j w_j.$$

Thus, in the pseudocode, the particle weight is determined by the **single most likely landmark correspondence** for the current observation.

In our implementation, however, a particle may receive multiple observations at the same time step,

$$Z_t = \{z_t^1, z_t^2, \dots, z_t^M\}.$$

For each observation (z_t^i), the association step still selects the most likely landmark,

$$L_i = \max_j p(z_t^i | x_t^{[k]}, \mu_j^{[k]}),$$

but instead of retaining only one likelihood for the particle, the evidence provided by all observations is accumulated. Assuming conditional independence of the measurements given the particle state and map,

$$p\left(Z_t \mid x_t^{[k]}, m^{[k]}\right) = \prod_{i=1}^M L_i.$$

Therefore, our implementation uses

$$w^{[k]} \propto \prod_{i=1}^M L_i$$

as the particle weight.

To improve numerical stability, this product is computed in the logarithmic domain:

$$\log w^{[k]} = \sum_{i=1}^M \log L_i.$$

This corresponds directly to the implementation on `fastslam.update_particle`:

```
for z in measurements:
    likelihood, landmark_idx = self.associate(particle, z)
    log_weight += np.log(likelihood)
```

After accumulating the log-likelihoods, the conventional particle weight is recovered as

$$w^{[k]} = \exp\left(\log w^{[k]}\right).$$

Hence, the main difference is that **the pseudocode defines the particle weight from the maximum likelihood associated with the current observation, whereas our implementation aggregates the likelihood contributions of all observations available at the current time step.** Under the conditional independence assumption, this corresponds to evaluating the joint measurement likelihood

$$p\left(Z_t \mid x_t^{[k]}, m^{[k]}\right) = \prod_{i=1}^M p\left(z_t^i \mid x_t^{[k]}, m^{[k]}\right).$$

Therefore, the resulting particle weight incorporates the consistency of the particle with the **complete measurement set** rather than relying solely on the most likely individual correspondence.

1.5.2 Experiment 1 — Low Motion Noise

```
[20]: low_noise_result = run_experiment(LOW_MOTION_NOISE, seed=42)
```

1.5.3 Experiment 2 — High Motion Noise

```
[21]: high_noise_result = run_experiment(HIGH_MOTION_NOISE, seed=42)
```

1.6 6. Ground Truth vs FastSLAM Estimated Trajectory

The two experiments are shown side by side using the same axes.

```
[22]: fig, axes = plt.subplots(
    1, 2,
    figsize=(14, 6),
    sharex=True,
    sharey=True
)

experiments = [
    ("Low motion noise", low_noise_result, LOW_MOTION_NOISE),
    ("High motion noise", high_noise_result, HIGH_MOTION_NOISE),
]

for ax, (title, result, noise) in zip(axes, experiments):
    desired = result["desired_poses"]
    gt = result["ground_truth"]
    est = result["estimated"]

    des_x = [p.x for p in desired]
    des_y = [p.y for p in desired]

    gt_x = [p.x for p in gt]
    gt_y = [p.y for p in gt]

    est_x = [p.x for p in est]
    est_y = [p.y for p in est]

    landmark_x = [lm.pos.x for lm in landmarks]
    landmark_y = [lm.pos.y for lm in landmarks]

    ax.scatter(
        landmark_x,
        landmark_y,
        marker="o",
        s=45,
        label="Landmarks"
    )

    ax.plot(
        des_x,
        des_y,
        linestyle=":",
        linewidth=2.0,
        label="Desired trajectory"
    )
```

```

ax.plot(
    gt_x,
    gt_y,
    linewidth=2.2,
    label="Ground truth"
)

ax.plot(
    est_x,
    est_y,
    linestyle="--",
    linewidth=2.0,
    label="FastSLAM estimate"
)

ax.scatter(
    gt_x[0],
    gt_y[0],
    marker="o",
    s=70,
    label="Start"
)

ax.scatter(
    gt_x[-1],
    gt_y[-1],
    marker="x",
    s=90,
    linewidths=2,
    label="End"
)

ax.set_title(
    f"{title}\n"
    f"Motion noise = ({noise[0]:.2f}, {noise[1]:.2f})"
)

ax.set_xlabel("x [m]")
ax.set_ylabel("y [m]")
ax.set_xlim(*x_lims)
ax.set_ylim(*y_lims)
ax.set_aspect("equal")
ax.grid(True)
ax.legend()

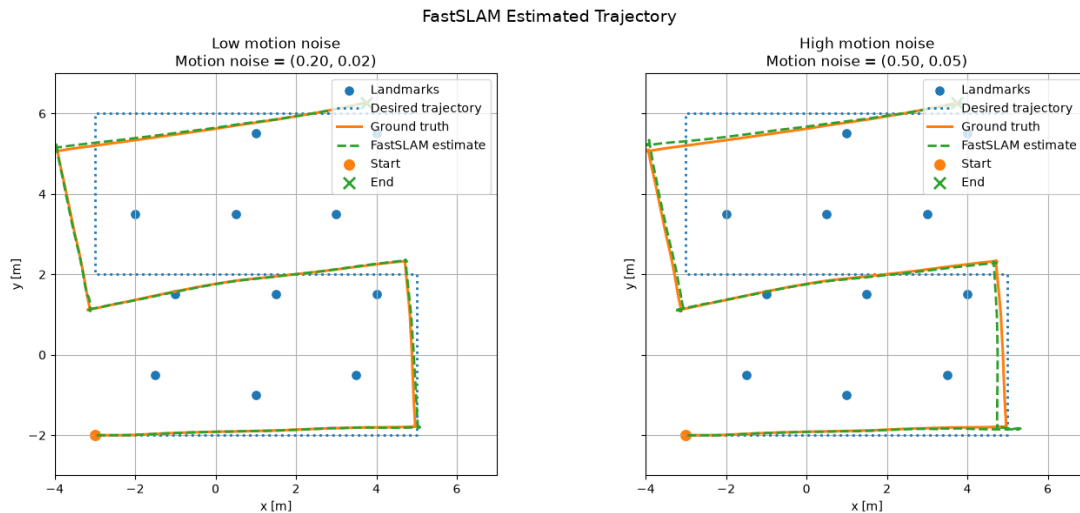
fig.suptitle(
    "FastSLAM Estimated Trajectory",

```

```

    fontsize=14
)
fig.tight_layout()
plt.show()

```



1.6.1 Trajectory Analysis

The estimated trajectory follows the ground-truth trajectory while exhibiting deviations caused by uncertainty in the sampled motion model. Increasing the motion-noise level increases the dispersion of the particle set, which directly affects the uncertainty of the estimated robot trajectory.

The comparison therefore illustrates the role of the particle set in representing uncertainty over the robot pose: lower motion uncertainty produces a more concentrated particle distribution, whereas higher motion uncertainty generates a broader set of trajectory hypotheses.

1.7 7. Position Error Comparison

The position error is computed as the Euclidean distance between the FastSLAM estimate and the corresponding ground-truth position.

```

[23]: low_error = position_error(low_noise_result)
      high_error = position_error(high_noise_result)

      time = np.arange(1, len(control_sequence) + 1) * dt

      fig, ax = plt.subplots(figsize=(10, 5.5))

      ax.plot(
          time,
          low_error,

```

```

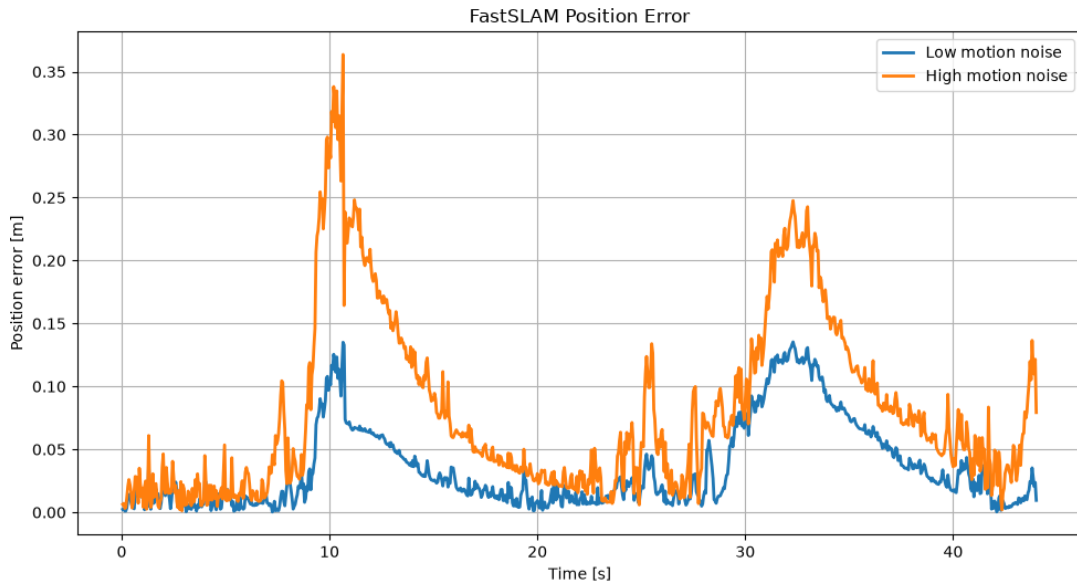
        linewidth=2,
        label="Low motion noise"
    )

    ax.plot(
        time,
        high_error,
        linewidth=2,
        label="High motion noise"
    )

    ax.set_title("FastSLAM Position Error")
    ax.set_xlabel("Time [s]")
    ax.set_ylabel("Position error [m]")
    ax.grid(True)
    ax.legend()

    fig.tight_layout()
    plt.show()

```



1.7.1 Error Analysis

The position error is defined as

$$e_t = \sqrt{(x_t - \hat{x}_t)^2 + (y_t - \hat{y}_t)^2}.$$

The comparison between the two experiments provides a quantitative measure of the influence of

motion uncertainty on the FastSLAM estimate. The error curves should be interpreted together with the particle dispersion, since increased motion uncertainty produces a wider range of pose hypotheses that must subsequently be corrected using landmark observations.

1.7.2 Quantitative Error Analysis

To complement the temporal error curves, the Root Mean Square Error (RMSE) is computed for each experiment:

$$\text{RMSE} = \sqrt{\frac{1}{T} \sum_{t=1}^T e_t^2}.$$

The RMSE provides a single measure of the overall position estimation error throughout the simulation, allowing the effect of motion uncertainty to be quantitatively compared between the two experiments.

```
[24]: low_rmse = np.sqrt(np.mean(low_error**2))
      high_rmse = np.sqrt(np.mean(high_error**2))

      print(f"Low motion noise RMSE: {low_rmse:.4f} m")
      print(f"High motion noise RMSE: {high_rmse:.4f} m")
```

Low motion noise RMSE: 0.0492 m

High motion noise RMSE: 0.1067 m

1.8 8. FastSLAM 2.0 Overview

FastSLAM 2.0 extends the FastSLAM 1.0 formulation by incorporating the current measurement into the proposal distribution used to sample the robot pose.

In FastSLAM 1.0, the new pose is sampled only from the motion model:

$$x_t^{[k]} \sim p(x_t | x_{t-1}^{[k]}, u_t).$$

Consequently, the particle prediction is driven exclusively by the control input and the motion uncertainty. The measurement is incorporated afterwards through landmark association and map updates.

FastSLAM 2.0 instead uses a proposal distribution that incorporates both the motion model and the current observation:

$$x_t^{[k]} \sim p(x_t | x_{t-1}^{[k]}, u_t, z_t).$$

The current measurement therefore provides information about the robot pose **before the particle is sampled**. This produces a proposal distribution that is more consistent with the available sensor observation.

Conceptually, the main difference between the two formulations is therefore the proposal distribution:

$$\boxed{\text{FastSLAM 1.0: } p(x_t \mid x_{t-1}, u_t)}$$

versus

$$\boxed{\text{FastSLAM 2.0: } p(x_t \mid x_{t-1}, u_t, z_t)}.$$

The landmark estimates are still maintained independently within each particle using EKF-based updates. Thus, the main modification introduced by FastSLAM 2.0 concerns the **proposal distribution for the robot pose**, rather than the fundamental representation of the landmark maps.

In the following section, both algorithms are evaluated using the same environment, landmark configuration, control sequence, sensor model, number of particles, and motion-noise levels.

1.9 9. FastSLAM 1.0 vs FastSLAM 2.0

FastSLAM 1.0 and FastSLAM 2.0 are evaluated under the same experimental conditions to provide a consistent comparison between the two algorithms. Both methods use the same simulation environment, landmark map, control sequence, sensor model, number of particles, and motion-noise levels.

The main difference between the two implementations is the proposal distribution used to sample the robot pose. FastSLAM 1.0 samples the new pose from the motion model, while FastSLAM 2.0 incorporates the current measurement into the proposal distribution.

The same simulation and analysis functions are reused for both algorithms. Therefore, the comparison is performed by changing only the FastSLAM implementation passed to `run_experiment`, while keeping the remaining experimental configuration unchanged.

```
[25]: ALGORITHMS = [
    ("FastSLAM 1.0", FastSLAM),
    ("FastSLAM 2.0", FastSLAM2),
]

NOISE_LEVELS = [
    ("Low", LOW_MOTION_NOISE),
    ("High", HIGH_MOTION_NOISE),
]

comparison_results = {}

for algorithm_name, algorithm_cls in ALGORITHMS:
    for noise_name, noise_value in NOISE_LEVELS:
        print(f"Running {algorithm_name} - {noise_name} motion noise ...")
        comparison_results[(algorithm_name, noise_name)] = run_experiment(
            noise_value,
            seed=42,
            algorithm_cls=algorithm_cls
        )
```

Running FastSLAM 1.0 - Low motion noise ...
Running FastSLAM 1.0 - High motion noise ...
Running FastSLAM 2.0 - Low motion noise ...
Running FastSLAM 2.0 - High motion noise ...

1.9.1 Trajectory Comparison

The four experiments below use the same ground-truth trajectory. The columns correspond to the motion-noise level, while the rows correspond to the FastSLAM variant.

```
[26]: fig, axes = plt.subplots(
    2, 2,
    figsize=(14, 12),
    sharex=True,
    sharey=True
)

landmark_x = [lm.pos.x for lm in landmarks]
landmark_y = [lm.pos.y for lm in landmarks]

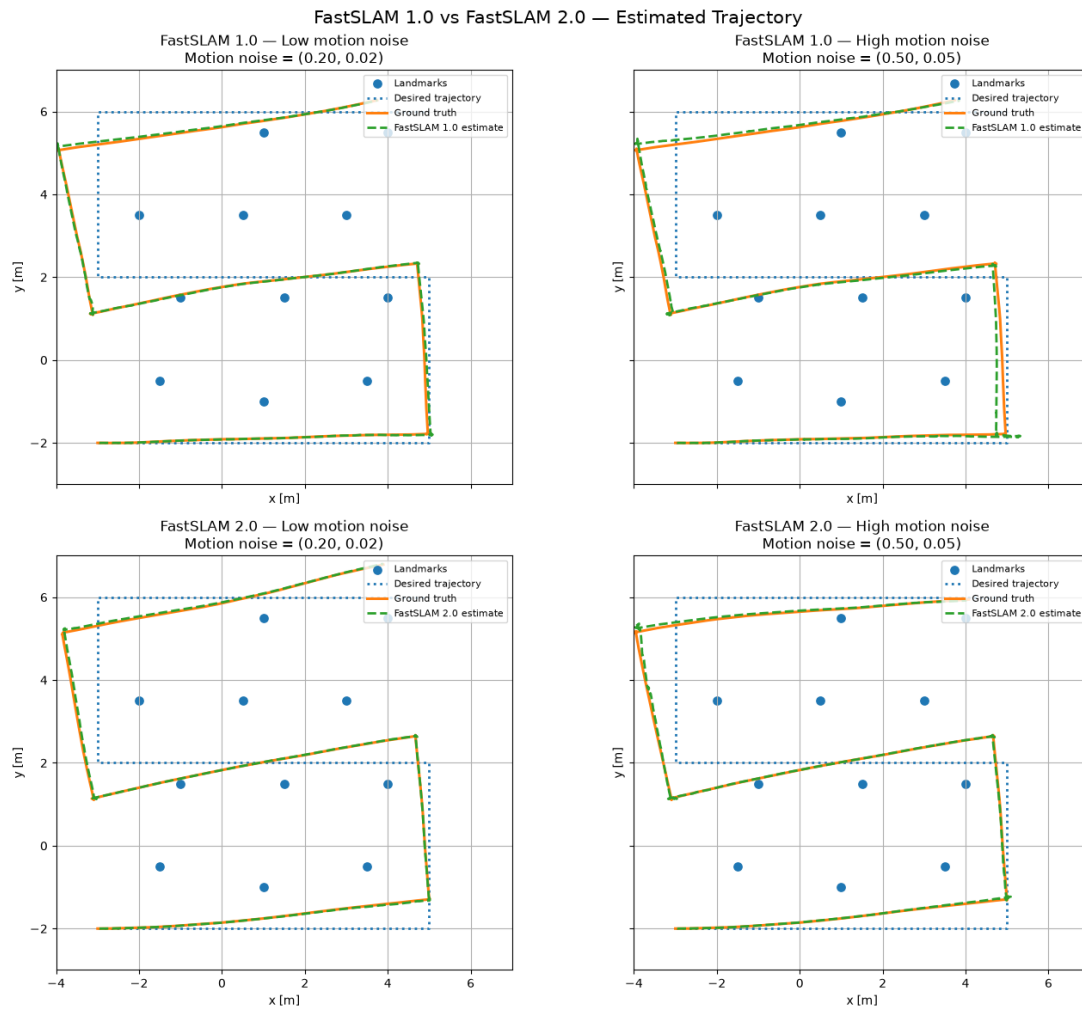
for row, (algorithm_name, _) in enumerate(ALGORITHMS):
    for col, (noise_name, noise_value) in enumerate(NOISE_LEVELS):
        ax = axes[row, col]
        result = comparison_results[(algorithm_name, noise_name)]

        desired = result["desired_poses"]
        gt = result["ground_truth"]
        est = result["estimated"]

        ax.scatter(landmark_x, landmark_y, marker="o", s=45, label="Landmarks")
        ax.plot([p.x for p in desired], [p.y for p in desired], linestyle=":",
        linewidth=2.0, label="Desired trajectory")
        ax.plot([p.x for p in gt], [p.y for p in gt], linewidth=2.2,
        label="Ground truth")
        ax.plot([p.x for p in est], [p.y for p in est], linestyle="--",
        linewidth=2.0, label=f"{algorithm_name} estimate")

        ax.set_title(
            f"{algorithm_name} - {noise_name} motion noise\n"
            f"Motion noise = ({noise_value[0]:.2f}, {noise_value[1]:.2f})"
        )
        ax.set_xlabel("x [m]")
        ax.set_ylabel("y [m]")
        ax.set_xlim(*x_lims)
        ax.set_ylim(*y_lims)
        ax.set_aspect("equal")
        ax.grid(True)
        ax.legend(fontsize=8)
```

```
fig.suptitle("FastSLAM 1.0 vs FastSLAM 2.0 - Estimated Trajectory", fontsize=15)
fig.tight_layout()
plt.show()
```



1.9.2 Position Error Comparison

For each experiment, the position error is evaluated using the Euclidean distance between the estimated and ground-truth positions.

```
[27]: time = np.arange(1, len(control_sequence) + 1) * dt

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5), sharex=True, sharey=True)
comparison_errors = {}
```

```

for col, (noise_name, noise_value) in enumerate(NOISE_LEVELS):
    ax = axes[col]

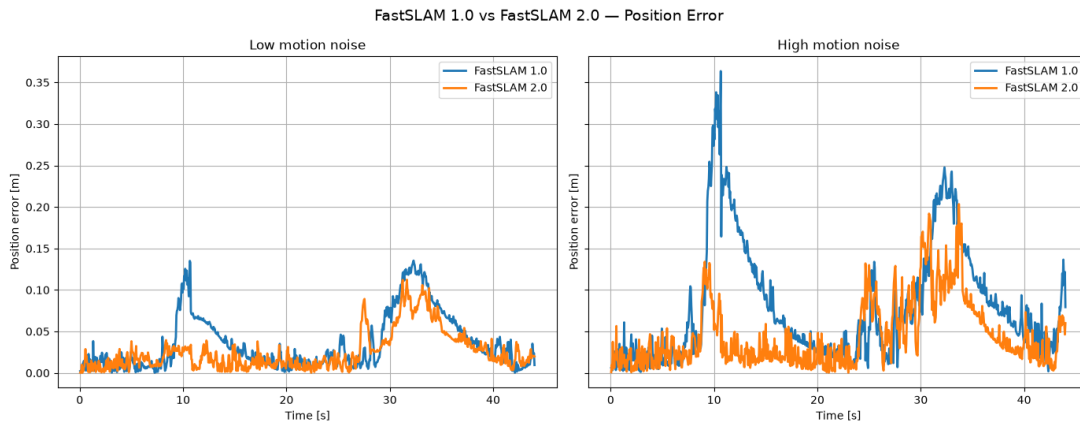
    for algorithm_name, _ in ALGORITHMS:
        error = position_error(comparison_results[(algorithm_name, noise_name)])
        comparison_errors[(algorithm_name, noise_name)] = error

        ax.plot(time, error, linewidth=2, label=algorithm_name)

    ax.set_title(f"{noise_name} motion noise")
    ax.set_xlabel("Time [s]")
    ax.set_ylabel("Position error [m]")
    ax.grid(True)
    ax.legend()

fig.suptitle("FastSLAM 1.0 vs FastSLAM 2.0 - Position Error", fontsize=14)
fig.tight_layout()
plt.show()

```



1.9.3 Summary Statistics

The following statistics summarize the position error and the effective number of particles for each experiment.

```

[28]: print(f"{'Algorithm':<15}{'Noise':<8}{'Mean err. [m]':<16}{'Max err. [m]':<15}{'RMSE [m]':<12}{'Median N_eff':<14}")
print("-" * 80)

for algorithm_name, _ in ALGORITHMS:
    for noise_name, _ in NOISE_LEVELS:
        error = comparison_errors[(algorithm_name, noise_name)]
        n_eff = comparison_results[(algorithm_name, noise_name)]["n_eff"]

```

```

rmse = np.sqrt(np.mean(error**2))

print(
    f"{algorithm_name:<15}{noise_name:<8}"
    f"{error.mean():<16.4f}{error.max():<15.4f}"
    f"{rmse:<12.4f}{np.median(n_eff):<14.1f}"
)

```

Algorithm	Noise	Mean err. [m]	Max err. [m]	RMSE [m]	Median N_eff
FastSLAM 1.0	Low	0.0357	0.1353	0.0492	46.7
FastSLAM 1.0	High	0.0803	0.3636	0.1067	39.9
FastSLAM 2.0	Low	0.0255	0.1123	0.0346	47.7
FastSLAM 2.0	High	0.0451	0.2033	0.0604	44.7

1.9.4 Interpretation

The position-error curves show that both algorithms are affected by increasing motion uncertainty, with larger error peaks occurring under high motion noise. However, the magnitude of these peaks differs between the two proposal strategies.

Under low motion noise, both FastSLAM variants maintain relatively small position errors for most of the trajectory. FastSLAM 2.0 generally follows a lower error profile, although both algorithms exhibit similar behavior during the main error increase around 30–35 s.

Under high motion noise, the difference becomes more pronounced. FastSLAM 1.0 exhibits a large error peak around 10 s, reaching approximately 0.36 m, while FastSLAM 2.0 remains substantially lower in the same interval. A similar behavior occurs around 30–35 s, where FastSLAM 1.0 reaches errors above 0.20 m, whereas FastSLAM 2.0 remains generally below this level.

These results are consistent with the different proposal distributions used by the two algorithms. FastSLAM 1.0 samples the robot pose using only the motion model, making the particle distribution more sensitive to motion uncertainty. FastSLAM 2.0 incorporates the current measurement into the proposal distribution, allowing the observation to influence the pose hypothesis before the particle is propagated.

The summary statistics complement the temporal error curves by quantifying the mean, maximum, and RMSE errors together with the median effective number of particles. Therefore, the comparison considers not only isolated error peaks, but also the overall estimation error and particle-filter behavior across the complete trajectory.

1.10 9. FastSLAM Animation

The same particle-filter evolution can be inspected dynamically for each motion-noise level.

```

[29]: def animate_experiment(result, title, frame_step=4):
        des = result["desired_poses"]
        gt = result["ground_truth"]
        est = result["estimated"]

```

```

particles = result["particles"]
particle_maps = result["particle_maps"]
n_eff = result["n_eff"]
measurements = result["measurements"]

frames = range(
    0,
    len(particles),
    frame_step
)
fig, ax = plt.subplots(figsize=(8, 7))

def update(frame_index):
    ax.clear()

    ax.scatter(
        [lm.pos.x for lm in landmarks],
        [lm.pos.y for lm in landmarks],
        marker="o",
        s=45,
        label="Ground truth landmarks"
    )

    current_map = particle_maps[frame_index]

    if current_map:
        ax.scatter(
            [v["mu"][0, 0] for v in current_map.values()],
            [v["mu"][1, 0] for v in current_map.values()],
            marker="x",
            s=65,
            linewidths=2,
            label="Estimated landmarks"
        )

    ax.plot(
        [p.x for p in gt[:frame_index + 2]],
        [p.y for p in gt[:frame_index + 2]],
        linewidth=2,
        label="Ground truth"
    )

    ax.plot(
        [p.x for p in des[:frame_index + 2]],

```

```

        [p.y for p in des[:frame_index + 2]],
        linestyle=":",
        linewidth=2,
        label="Desired trajectory"
    )

    ax.scatter(
        [p.x for p in particles[frame_index]],
        [p.y for p in particles[frame_index]],
        s=18,
        alpha=0.55,
        label="Particles"
    )

    ax.plot(
        [p.x for p in est[:frame_index + 1]],
        [p.y for p in est[:frame_index + 1]],
        linestyle="--",
        linewidth=2,
        label="FastSLAM estimate"
    )

    current_gt = gt[frame_index + 1]
    ax.scatter(
        current_gt.x,
        current_gt.y,
        marker="x",
        s=90,
        linewidths=2,
        label="Robot"
    )

    ax.set_xlim(*x_lims)
    ax.set_ylim(*y_lims)
    ax.set_aspect("equal")

    ax.set_xlabel("x [m]")
    ax.set_ylabel("y [m]")

    ax.set_title(
        f"{title} - t = {frame_index * dt:.2f} s"
    )

    ax.grid(True)

```

```

ax.legend(loc="upper right")

ax.text(
    0.02,
    0.02,
    f"N particles: {N_PARTICLES}\n"
    f"N observed landmarks: {len(current_map)}\n"
    f"N detected landmarks: "
    f"{len(measurements[frame_index])}\n"
    f"$N_{{eff}}$: {n_eff[frame_index]:.1f}",
    transform=ax.transAxes,
    verticalalignment="bottom",
    bbox=dict(
        boxstyle="round",
        facecolor="white",
        alpha=0.8
    )
)

ani = FuncAnimation(
    fig,
    update,
    frames=frames,
    interval=50,
    repeat=False,
    blit=False
)

os.makedirs("img", exist_ok=True)
filename = title.lower().replace(" ", "_")

ani.save(
    f"img/{filename}.gif",
    writer="pillow",
    fps=20
)

plt.close(fig)

display(
    HTML(ani.to_jshtml())
)

return ani

```

1.10.1 Low Motion Noise

```
[30]: animate_experiment(low_noise_result, "FastSLAM - Low Motion Noise")
```

```
<IPython.core.display.HTML object>
```

```
[30]: <matplotlib.animation.FuncAnimation at 0x26f94635390>
```

1.10.2 High Motion Noise

```
[31]: animate_experiment(high_noise_result, "FastSLAM - High Motion Noise")
```

```
<IPython.core.display.HTML object>
```

```
[31]: <matplotlib.animation.FuncAnimation at 0x26fb44dde90>
```

1.10.3 FastSLAM 2.0 — Low Motion Noise

```
[33]: animate_experiment(  
    comparison_results[("FastSLAM 2.0", "Low")],  
    f"FastSLAM 2.0 - Low Motion Noise ({LOW_MOTION_NOISE[0]:.2f},  
    ↪{LOW_MOTION_NOISE[1]:.2f})"  
)
```

```
<IPython.core.display.HTML object>
```

```
[33]: <matplotlib.animation.FuncAnimation at 0x26f9c176010>
```

1.10.4 FastSLAM 2.0 — High Motion Noise

```
[32]: animate_experiment(  
    comparison_results[("FastSLAM 2.0", "High")],  
    f"FastSLAM 2.0 - High Motion Noise ({HIGH_MOTION_NOISE[0]:.2f},  
    ↪{HIGH_MOTION_NOISE[1]:.2f})"  
)
```

```
<IPython.core.display.HTML object>
```

```
[32]: <matplotlib.animation.FuncAnimation at 0x26f99e42f10>
```

1.11 10. Conclusion

This work provided a practical implementation and evaluation of FastSLAM 1.0 and FastSLAM 2.0 as Rao-Blackwellized particle-filter approaches for simultaneous localization and mapping. In both algorithms, the robot pose is represented by a particle set, while the map associated with each particle is maintained through individual EKF landmark estimates.

The experiments demonstrated the influence of motion uncertainty on localization performance. As the motion noise increased, larger position errors and stronger variations in the particle distribution were observed. The comparison between FastSLAM 1.0 and FastSLAM 2.0 also illustrated the importance of the proposal distribution. FastSLAM 1.0 samples the robot pose using only the motion model, whereas FastSLAM 2.0 incorporates the current measurement into the proposal distribution, allowing the observation to influence the pose hypothesis before resampling.

This difference became particularly relevant under higher motion uncertainty, where FastSLAM 1.0 exhibited larger position-error peaks, while FastSLAM 2.0 generally maintained a more consistent estimation. Under lower motion noise, the difference between the two approaches was less pronounced, since the motion model already provided a more constrained pose prediction.

The implementation also demonstrated the role of measurement likelihoods, Mahalanobis distance, data association, and EKF landmark updates in evaluating and maintaining the hypotheses represented by the particles. Overall, the experiments illustrate how the choice of proposal distribution affects particle efficiency and localization accuracy, providing a practical understanding of the main algorithmic difference between FastSLAM 1.0 and FastSLAM 2.0.